文档类型	学习讲义（自学用）	版　　本	2.0
适用对象	零基础，需高中物理与基础代数	配套项目	acoustic-lab/（可运行代码）
内容来源	韩纪庆等《语音信号处理（第 3 版）》；王泉《声纹技术：从核心算法到工程实践》；项目实测数据		
在线仓库	github.com/Lzq2190/voiceprint-lab（讲义、代码、图表与数据；仓库说明见第 0 讲）		
说　　明	书中公式按标准形式重写并标注出处；全部图表由配套脚本实测生成，非示意图		

信 号 处 理 与 识 别 系 列

声纹信号处理

从零开始的学习讲义

信号 → 特征 → 神经网络 → 声纹识别

共 16 讲 · 每讲含目标、产出、做法、误区与自检

配套复现项目 acoustic-lab　·　全部图表为脚本实测生成

github.com/Lzq2190/voiceprint-lab

2026 年 11 月

目　录

共 17 讲 · 页码为正文页码

第 0 讲 先读这个：怎么用这份讲义 …… 1

第 1 讲 声音是什么：从声波到正弦波 …… 5

第 2 讲 数字化与分帧加窗：把声音变成"能算的数据" …… 9

第 3 讲 时域特征：短时能量、过零率、自相关 …… 14

第 4 讲 傅里叶变换：从时间到频率 …… 18

第 5 讲 时频分析：STFT 与不确定原理 …… 22

第 6 讲 MFCC：最常用的声学特征 …… 25

第 7 讲 小波变换与 EMD：超越固定窗的时频分析 …… 30

第 8 讲 神经网络基础：BP 神经网络 …… 35

第 9 讲 卷积神经网络 CNN …… 40

第 10 讲 训练、评估与过拟合 …… 44

第 11 讲 声纹识别概览：任务、流程、与语音识别的区别 …… 48

第 12 讲 声纹评价指标：EER、ROC、DCF …… 51

第 13 讲 经典声纹方法：GMM、UBM、i-vector …… 55

第 14 讲 深度学习声纹：x-vector 与损失函数 …… 58

第 15 讲 收尾：学习路线回顾与进阶方向 …… 61

附录 中英术语对照、章节映射与参考文献 …… 63

每一讲均按「目标 → 产出 → 准备 → 怎么做 + 为什么 → 误区与注意事项 → 自检」编排。自检项可直接勾选，用于确认是否真正掌握；配套可运行项目 acoustic-lab/ 中的脚本生成了本文全部图表与数值。

第 0 讲

先读这个：怎么用这份讲义

目标

- 明白这份讲义和"看教材"的区别：它把每一步都摊开写，不留"读者自明"的跳跃。
- 知道 16 讲之间的依赖关系，能判断自己卡在哪一层。
- 把运行环境跑通，能执行 `python run.py --check` 并看到验证数字。

产出

- 一条能跑通的命令，和一张自己画出的学习进度表。
- 对"我到底要学会什么"有一句话的回答。

为什么很多教程看不懂

初学者读信号处理教材最常卡在三处，这份讲义专门针对它们：

第一，公式没有落到数字。书上写 `Δf = Fs/N`，读者记住了符号，却不知道"8 kHz 采样、256 点 FFT，频率分辨率就是 31.25 Hz"这个具体结论。本讲义每个公式后面都跟一个带数字的算例，你要能自己把这个数字算出来。

第二，默认你会。教材写"对信号做 Mel 滤波"，默认你知道滤波器组是一堆三角形、默认你知道为什么要取对数。本讲义把这些"默认"全部展开，包括看起来傻的问题：为什么是三角形不是矩形？为什么取 log 不取平方根？

第三，只讲"是什么"，不讲"错了会怎样"。没有反例的知识不牢固。每一讲后面都有"误区"和"破坏性实验"：改一个参数，先预测结果，再运行验证。这个"预测—打脸"的过程才是真正记住的地方。

三条学习原则

原则 1：先跑通，再抠懂 每个算法先在 `acoustic-lab` 里跑出图、跑出数值，让眼睛先"认识"它，再回头推公式。你会发现很多"看不懂的公式"，跑一次图就懂了——因为公式描述的是一件很具体的事。

原则 2：每个算法走"四步闭环" ① 读本讲对应段落，把符号和物理意义对上；② 不看现成代码，自己用 20–30 行 numpy 写一遍；③ 对拍（和 `numpy.fft`、数值梯度、重构误差比）；④ 破坏性实验（改参数，先预测再验证）。第 ③ 步是硬门槛：算法对不对不靠感觉，靠 1e−12 这种量级。

原则 3：别急着上框架 `acoustic-lab` 里的 BPNN 和 CNN 是纯 numpy 手写的——它们就是 PyTorch 替你做的全部事情。等你能自己写梯度检查，再上框架是半天的事；反过来先学框架，就永远只会调 API，遇到 loss 不降只能干瞪眼。

16 讲的依赖关系

表 0-1　16 讲的依赖关系

层	讲次	学完要能回答的问题
第 1 层：信号	第 1–5 讲	一段声音怎么变成数字？怎么变成频谱？为什么时间和频率不能同时精确？
第 2 层：特征	第 6–7 讲	为什么声纹几乎都用 MFCC？小波和 EMD 补上了什么？
第 3 层：模型	第 8–10 讲	神经网络怎么从数据里学规律？为什么不能只看准确率？
第 4 层：声纹	第 11–14 讲	怎么判断"是不是同一个人/同一种工况"？EER 为什么是 2 倍？
收口	第 15–16 讲	整条链路怎么串起来？下一步往哪走？

依赖是硬的：第 6 讲的 MFCC 建立在第 4、5 讲的 FFT 与 STFT 上；第 9 讲的 CNN 建立在第 8 讲的反向传播上。哪一讲读不下去，往回退一讲，不要硬扛。

环境准备

```
# 只需要 Python 3 + 两个库，其余全部手写
pip install numpy pillow

# 进入项目目录，跑通冒烟测试（约 10 秒）
cd acoustic-lab
python run.py --check
```

两个库的分工：`numpy` 只用来做矩阵运算和数学函数（`np.fft` 仅作为"标准答案"用来对拍，我们自己写的算法不调用它）；`pillow` 只用来画图。也就是说，书上的每个算法都在这个项目里被重新实现了一遍，你可以逐行对照。

> **验证装好了** 运行 `python -c "import numpy, PIL; print(numpy.__version__)"`，能打印出版本号即可。若报错，通常是装到了另一个 Python 解释器里——用 `python -m pip install numpy pillow` 保证装到同一个解释器。

材料从哪来：GitHub 仓库

这份讲义以及它引用的全部代码、图表、数据都在一个公开仓库里，不用自己敲。地址是：

```
https://github.com/Lzq2190/voiceprint-lab
```

第一次使用，克隆到本地：

```
git clone https://github.com/Lzq2190/voiceprint-lab.git
cd voiceprint-lab
```

不想装 git 也可以：打开上面那个网址，点绿色 `Code` 按钮 → `Download ZIP`，解压后的目录结构和 clone 完全一样。

表 0-2　仓库里各目录放的是什么

路径	内容	怎么用
`acoustic-lab/`	手写算法与复现项目：DFT/FFT、小波、EMD、MFCC、BPNN/CNN、零依赖绘图引擎	进去执行 `python run.py`
`acoustic-lab/data/`	6 段 8 kHz 合成变压器声纹（正常、铁芯松动、直流偏磁、局部放电、短路冲击、风机老化）	可用 `--wav` 换成自己的录音
`acoustic-lab/outputs/`	57 张图、7 个 CSV、`metrics.json`	讲义里的每一张图都来自这里
`guide-v2/index.html`	本讲义的网页版，离线可开，学习进度存在浏览器里	双击打开即可
`guide-v2/pdf/声纹信号处理学习讲义.pdf`	本讲义的 A4 打印版	阅读、打印、装订
`guide-v2/tools/`	重绘插图（`render_light_figs.py`）与重新排版 PDF（`build_pdf.py`）的脚本	改过正文后重跑这两个

> **保持最新**　讲义还在修订时，在仓库目录执行 `git pull` 就能拿到新版；如果本地自己也改过文件而提示冲突，先 `git stash` 再拉，拉完 `git stash pop`。

> **仓库里故意没有的东西**　两本参考教材（韩纪庆《语音信号处理（第 3 版）》、王泉《声纹技术》）的原文与提取文本没有上传——那是版权内容，请自备正版书。仓库只放自写的代码、脚本生成的图表和自撰的讲义正文，所以你不必依赖这两本书也能读完每一讲。

如果你只想跑代码、暂时不读讲义，clone 之后直接 `cd acoustic-lab` 再执行 `python run.py --check`，十秒内会打印出 DFT/FFT/小波重构/EMD/梯度检查的数值验证表。那张表是判断"代码有没有被改坏"的唯一依据，比看任何说明都可靠。

数据说明：为什么用合成声纹

讲义里的实验用 6 类合成变压器声纹（正常、铁芯松动、直流偏磁、局部放电、短路冲击、风机老化），采样率 8 kHz、时长 0.5 s。合成不是为了偷懒，而是因为合成数据有标准答案：我知道第 n 个采样点应该等于多少，才能验证你的 FFT、你的小波重构、你的梯度推导到底对不对。

真实录音的入口也留好了：`python run.py --wav 你的文件.wav`。换真实录音后你会发现准确率不再漂亮——这恰恰是重点，第 10、15 讲会反复讲这件事：合成数据上的 **100%** 只证明实现正确，不证明现场可用。

建议节奏

表 0-3　建议的四周学习节奏

周	内容	每周产出（自检用）
第 1 周	第 1–5 讲	能手写 DFT、画出语谱图、说清不确定原理

周	内容	每周产出（自检用）
第 2 周	第 6–7 讲	能手写 MFCC 全流程、能解释小波与 EMD 的取舍
第 3 周	第 8–10 讲	能手写 BPNN/CNN 并通过梯度检查、会看混淆矩阵
第 4 周	第 11–15 讲	能对一份识别结果说出 EER 与 DCF、能判断论文结论的可信度

自检

- [ ] 我能说出这份讲义与教材的三点区别（有数字、不默认、讲反例）。
- [ ] 我已经跑通 `run.py --check` 并看到验证数值。
- [ ] 我知道 16 讲分四层，并能说出自己现在处于哪一层。
- [ ] 我已经把仓库克隆到本地（或解压了 ZIP），并跑出 `run.py --check` 的验证表。

第 1 讲

声音是什么：从声波到正弦波

目标

- 能写出正弦波 `x(t)=A·sin(2πft+φ)`，并说清 A、f、φ 三个量各决定听感的什么。
- 理解"复合音 = 基频 + 倍音叠加"，能解释为什么不同设备/不同人声听起来不同。
- 知道声纹诊断在物理上到底在比什么。

产出

- 用 numpy 合成一个 100 Hz 复合音，并打印出前 10 个采样值。
- 一张"物理量 → 主观感受"的对照表，能自己讲一遍。

先修知识与准备

需要：正弦函数、角度与弧度的区别（1 圈 = 2π 弧度）、`numpy` 的 `sin/arange`。不需要任何信号处理基础，也不需要复数（复数在第 4 讲才出现）。

① 声音是空气的压强波

声带、铁芯、风机叶片振动，推动空气一疏一密地传播，耳朵接收到的是压强随时间的变化。所以声音最本质的数学表示，就是一个随时间变化的量：

> x(t) = 某一时刻 t 的声压（单位 Pa，实际常用相对于 20 μPa 的分贝表示）

三个量级要记住，用来判断"这个采样率够不够"：人耳可听范围约 **20 Hz – 20 kHz**；人说话的主要能量在 **60–4000 Hz**（所以电话用 8 kHz 采样就够）；变压器本体噪声主要能量在 **50–2000 Hz**，其中 100 Hz（两倍工频）是磁致伸缩的基频。

> **为什么变压器是 100 Hz 而不是 50 Hz** 铁芯的磁致伸缩（硅钢片随磁场伸缩）与磁感应强度平方成正比，而 B 又以 50 Hz 交变，平方之后频率翻倍——所以本体噪声的基频是 100 Hz。这一点在第 4 讲的频谱图和文献里会反复出现（例如"50 Hz 倍频倒谱系数"就是用这个物理事实设计的特征）。

② 三个物理量，对应三种听感

最简单的声波是正弦波（纯音，音叉近似）：

> $x(t) = A \cdot \sin(2\pi \cdot f \cdot t + \varphi)$
>
> A：振幅（Pa）→ 决定响度（多大声）
>
> f：频率（Hz）→ 决定音高（多高）
>
> φ：初相位（弧度）→ 单听一个纯音听不出来，但多频率叠加时决定波形形状

为什么写成 `2πf` 而不是 `f`？因为正弦的自变量是角度，不是时间。一秒转 f 圈，一圈 2π 弧度，所以角速度 ω = 2πf（rad/s）。周期与频率互为倒数：`T = 1/f`。

> **手算一次，比看十遍公式有用** f = 100 Hz → 周期 T = 1/100 = **0.01 s = 10 ms**。若采样率 Fs = 8000 Hz，则一个周期内有 8000 × 0.01 = **80** 个采样点。记住这个"80 点/周期"，第 2 讲算帧长、第 4 讲算频率分辨率都要用它。

③ 真实声音都是复合音

纯音只有一个频率，但真实语音、乐器、变压器噪声都是很多正弦叠加：一个基频 f_0，加上频率是它整数倍的"倍音"（谐波）。

> $x(t) = A_1 \cdot \sin(2\pi \cdot f_0 \cdot t + \varphi_1) + A_2 \cdot \sin(2\pi \cdot 2f_0 \cdot t + \varphi_2) + A_3 \cdot \sin(2\pi \cdot 3f_0 \cdot t + \varphi_3) + \dots$
>
> 其中 f_0 是基频，$2f_0$、$3f_0$… 是倍音（谐波），A_k 是第 k 次谐波的振幅

关键结论：音色由谐波比例决定，不由基频决定。钢琴和小提琴弹同一个音（基频相同），听起来完全不同，因为它们的 2 次、3 次…谐波振幅比例不同。这条结论是"用频谱做识别"的全部依据：如果音色只由基频决定，那只需测一个频率就够了，用不着 FFT。

三种常见波形的谐波构成

表 1-1　三种常见波形的谐波构成

波形	谐波构成	听感	在声纹里的对应
纯正弦	只有 f_0	单薄、像音叉	正常运行的铁芯本体声（近似）
方波（近似）	f_0, $3f_0$, $5f_0$…（奇次），振幅按 1/k 衰减	空洞、像单簧管	铁芯松动后出现的奇次谐波增强
锯齿（近似）	所有次谐波，振幅按 1/k 衰减	明亮、像弦乐	直流偏磁导致的谐波整体抬升

④ 声纹诊断在物理上比的是什么

把上面的逻辑接到设备上：设备状态改变 → 振动力学结构改变 → 谐波结构或瞬态成分改变 → 声音的频谱/时频图改变。所以"声纹"不是玄学，它比的是这几件事：

表 1-2　典型故障的物理机制与声纹表现

故障	物理机制	声纹上的表现
铁芯松动	压紧力下降，片间摩擦与撞击	100 Hz 及其倍频能量变化，叠加周期性敲击脉冲
直流偏磁	磁化曲线不对称，半波饱和	出现偶次谐波与 50 Hz 倍频族（文献里用"50 Hz 倍频倒谱系数"专抓这个）
局部放电	微小放电产生宽频脉冲	时频图上出现稀疏的竖直亮条（高频、极短）
风机老化	轴承磨损、转速下降	宽带噪声抬升，高频能量比例上升
短路冲击	绕组受巨大电动力	突发冲击声，短时能量骤增

这张表是本讲义后半部分所有特征设计的"物理依据"：第 3 讲的过零率抓高频，第 6 讲的 MFCC 抓谐波包络，第 7 讲的小波抓瞬态，都是因为它们各自对应上表的一行。

⑤ 从 x(t) 到 x[n]：离散化的预告

计算机存不下连续函数，只能存一串数。做法是每隔 `Ts = 1/Fs` 秒取一个值：

> $x[n] = x(n \cdot Ts) = x(n / Fs)$，$n = 0, 1, 2, \ldots$
>
> 例：$Fs = 8000$ Hz、时长 0.5 s → 共 $8000 \times 0.5 = 4000$ 个采样点

这 4000 个数就是你后面所有操作的原材料。打印出来看，它只是一个上下波动的数列——所有的"频率""音色""故障"，都藏在这个数列的变化规律里，需要后面的工具才能挖出来。

动手 1：合成一个复合音并"看"它

```
import numpy as np

fs = 8000                       # 采样率 8 kHz
t = np.arange(fs) / fs          # 1 秒，共 8000 个时刻
f0 = 100                        # 基频 100 Hz（变压器本体噪声基频）

# 基频 + 2、3 次谐波，振幅递减（符合真实声音"谐波越高能量越低"）
x = 1.0*np.sin(2*np.pi*f0*t) + 0.5*np.sin(2*np.pi*2*f0*t) + 0.3*np.sin(2*np.pi*3*f0*t)

print(x[:10])                   # 前 10 个采样值
print('长度', len(x), '最大幅值', round(np.abs(x).max(), 3))
```

运行后你会看到类似 `[0. 0.15 0.29 ...]` 的一串小数，长度 8000，最大幅值约 1.8（三个分量在某个时刻同相叠加的结果，恰好大于 1）。这个"大于 1"很重要：叠加后幅值会超出单个分量，所以后面做 int16 量化时要注意限幅，否则会削波。

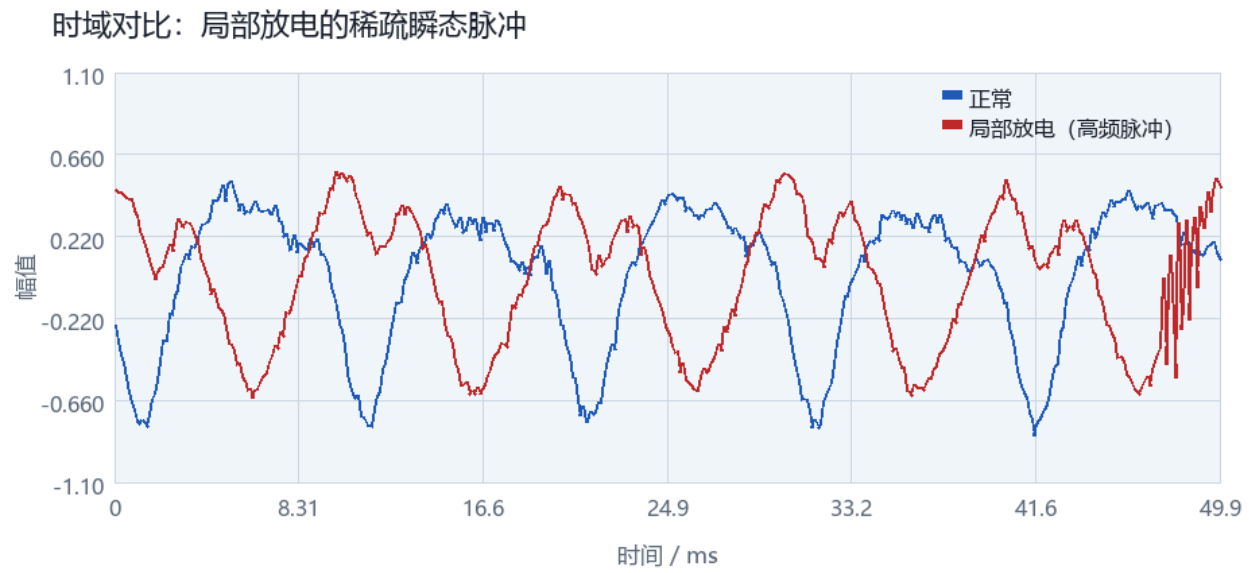


图 1-1　正常与局部放电的时域波形对比：正常样本是连续周期波动，局放样本在平缓背景上突然出现尖脉冲——只看波形就能分辨一类故障

误区与注意事项

误区 ① 把"频率高"当成"声音大"。频率和振幅是两个独立维度：高频可以很小声，低频可以很大声。② 以为基频决定音色。决定音色的是谐波比例。③ 把"采样点数多"当成"信息多"。采样率不够时，点再多也只是把混叠后的假信号画得更细（第 2 讲会算给你看）。

自检

- [ ] 我能写出 `x(t)=A·sin(2πft+φ)`，并说出 A、f、φ 各影响什么。
- [ ] 我能解释：为什么钢琴和小提琴弹同一个音听起来不一样？
- [ ] 我能算出 100 Hz 正弦在 8 kHz 采样下一个周期有 80 个点。
- [ ] 我能说出变压器本体噪声基频为什么是 100 Hz 而不是 50 Hz。

第 2 讲

数字化与分帧加窗：把声音变成"能算的数据"

目标

- 理解采样定理，并能自己算出一个"混叠"例子。
- 会用 6 dB/bit 估算量化精度，能解释为什么现场用 16 bit。
- 能把一段语音切成帧并加窗，说清 25 ms / 10 ms 这两个数字的来历。

产出

- 一段分帧加窗的代码，输出形状为 (帧数, 帧长)。
- 一张"采样率 × 帧长 → 点数"的对照表，能现场算。

先修知识与准备

需要：第 1 讲的正弦表示、对数运算（dB 的定义）、能看懂 `for` 循环和切片。本节不需要傅里叶知识。

① 采样：为什么必须"两倍"

采样就是把连续信号每隔 Ts 秒"拍一张照片"。问题在于：如果拍得太稀，快慢不同的两个信号会拍出完全一样的照片，你就再也分不清了。这叫混叠（aliasing）。

采样定理（奈奎斯特）说：要无失真地恢复最高频率为 f_max 的信号，采样率必须满足

> $Fs > 2 \cdot f_{max}$　　（$2 \cdot f_{max}$ 称为奈奎斯特频率）
> 反过来：能正确表示的最高频率只有 Fs/2，称为"奈奎斯特上限"

手算一个混叠例子：有一个 1000 Hz 的正弦，用 Fs = 800 Hz 采样（明显不够）。采样后它看起来是多少 Hz？混叠频率的算法是

> f_alias = |f − k·Fs| 中落在 [0, Fs/2] 的那个值
> 取 k = 1：|1000 − 800| = 200 Hz → 落在 [0, 400] 内，所以看起来是 200 Hz
> 结论：一个 1000 Hz 的信号，用 800 Hz 采样，会被误认成 200 Hz —— 而且再怎么增加采样点数也救不回来

这就是为什么"采样率"是设备选型的第一参数：一旦定了 **8 kHz**，**4000 Hz** 以上的成分注定是假的——不是"变模糊"，而是原封不动地折叠成低频混进来，事后任何算法都分不开。反过来，也不是越高越好——第 0 讲提过，16 kHz 之后识别率几乎不再提升，纯属浪费存储和算力。

4000 Hz 是理论值，工程上还要打折 8 kHz 的奈奎斯特上限是 Fs/2 = **4000 Hz**，这是"能表示"的极限，不是"能用"的带宽。要真做到不失真，采样之前必须先用抗混叠低通把 4000 Hz 以上滤掉；而真实滤波器不是悬崖，它有过渡带——一般从 3400 Hz 就开始滚降，到 4000 Hz 才压够，所以保护带要留 10%–20%。工程上把 **8 kHz** 的可用带宽认作 **3.4 kHz** 左右。电话语音标准之所以是 300–3400 Hz，就是因为要给这个滤波器留出过渡带，而不是因为 3400 Hz 以上没用。

所以选型该这样问 不是问"我要用的最高频率是多少"，而是问"最高频率 × 1.2 是多少，哪个采样率超过它"。比如要分析 3 kHz 的局放超声成分，8 kHz 只剩 3.4 kHz 的可信带，已经贴着边了；要看 4 kHz 以上，必须上 16 kHz。

常见采样率与它的"能力上限"

表 2-1　常见采样率与它的能力上限

采样率	奈奎斯特上限	典型用途	1 秒的数据量（16 bit 单声道）
8 kHz	4000 Hz	电话语音、变压器声纹常用	16000 字节 ≈ 15.6 KB
16 kHz	8000 Hz	语音识别标准（性价比顶点）	31.25 KB
44.1 kHz	22050 Hz	音乐 CD	86.1 KB

② 量化：一个比特值多少分贝

采样只离散了时间，振幅还是连续的。量化把振幅切成有限个等级：B 个比特 → **2^B** 个等级。16 bit → 65536 级，把 −1 到 +1 的电压范围分完，每一级约 3.05×10^{-5}。

量化的误差可以看成均匀分布在 ±q/2 之间的随机噪声（q 是级距），其功率为 $q^2/12$。信号是满幅正弦时功率为 $A^2/2$，两者一比得到著名的经验公式：

$$\mathrm{SNR(dB)} \approx 6.02\cdot B + 1.76$$

$B = 16 \rightarrow 6.02\times16 + 1.76 \approx 98$ dB

$B = 8 \rightarrow 6.02\times8 + 1.76 \approx 50$ dB

每多 1 bit，信噪比提高约 6 dB —— 这就是"一个比特 6 dB"的来历

拿它跟需求比：语音的动态范围约 55 dB，所以 10 bit（62 dB）以上就够；现场设备用 16 bit（98 dB）是为了留出余量——传声器前置放大器的本底噪声、突发冲击声（短路冲击）的峰值，都需要这 40 dB 的余量。

一个容易踩的坑 int16 的取值范围是 **−32768 ~ +32767**，不是 ±32768。所以定点转换要用 `np.clip(x, -1, 1) * 32767`，不要乘 32768；否则 +1.0 会溢出成 −32768，波形上出现一个突然的"反向尖峰"，后面所有特征都会被它污染。

采样与量化的前 14 个点（真实数据）

下面是 `acoustic-lab` 里"正常"样本的实际数据，第四列就是写进 WAV 文件的整数。你可以看到浮点值乘 32767 后取整的过程：

表 2-2　采样与量化的前 14 个点（fs = 8000 Hz，16 bit 量化）

n	t = n•Ts / ms	x[n]（浮点）	int16 量化
0	0.0000	-0.255686	-8378
1	0.1250	-0.346122	-11341
2	0.2500	-0.418829	-13723
3	0.3750	-0.463055	-15172
4	0.5000	-0.537507	-17612
5	0.6250	-0.617214	-20224
6	0.7500	-0.672789	-22045
7	0.8750	-0.711063	-23299
8	1.0000	-0.768570	-25183
9	1.1250	-0.784598	-25708
10	1.2500	-0.765243	-25074
11	1.3750	-0.779919	-25555
12	1.5000	-0.796850	-26110
13	1.6250	-0.736307	-24126

③ 分帧：因为语音是"短时平稳"的

一整句话从"开"到"始"，声学特性完全不同（这叫非平稳）。但发声器官的肌肉运动比声波振动慢得多，所以在**10–30 ms** 这么短的时间窗内，可以近似认为特性不变——这就是"短时平稳假设"，也是所有短时分析的前提。

于是我们把长序列切成一段段"帧"，每帧单独分析。两个参数：

> 帧长 L（frame length）：一帧包含多少个采样点，常用 20–30 ms
>
> 帧移 S（frame shift / hop）：相邻两帧起点间隔，常用 10 ms
>
> 重叠 = L − S；50% 重叠即 S = L/2

把毫秒换成点数（这一步初学者最常算错，务必自己算一遍）：

> 点数 = 时长(ms) × Fs / 1000
>
> 16 kHz、25 ms → 25 × 16000 / 1000 = 400 点
>
> 16 kHz、10 ms → 160 点
>
> 8 kHz、25 ms → 200 点
>
> 8 kHz、10 ms → 80 点

帧数怎么算：长度 N 的序列，帧长 L，帧移 S，不补零时

> 帧数 = floor((N − L) / S) + 1
>
> 例：N = 16000（1 秒 @16 kHz），L = 400，S = 160
>
> → (16000 − 400)/160 + 1 = 97.5 → 取整 97，再 +1 = 98 帧

98 帧这个数字很关键：1 秒语音原本 16000 个数，变成 98 个"帧"。如果每帧再压成 13 维 MFCC，就只剩 98×13 ≈ 1274 个数——数据量降了一个数量级，而判别信息基本保留。这就是"特征提取"存在的意义。

④ 加窗：为什么不能直接"切一刀"

分帧在数学上等于把信号乘上一个"矩形窗"（窗内为 1，窗外为 0）。而时域相乘 = 频域卷积，所以每一帧的频谱都会被矩形窗的频谱"抹开"：原本一根干净的谱线，会变成主瓣+一串旁瓣，能量泄漏到邻近频率上。

矩形窗的旁瓣只衰减到 −13 dB，泄漏很严重。换成两端平滑下降到 0 的窗（汉宁、汉明），相当于让帧的两端"淡入淡出"，截断处的突变消失，旁瓣就能压到 −31 ~ −43 dB。代价是主瓣变宽——频率分辨率变差。这是一个必须接受的折中。

常用窗函数对比

表 2-3　常用窗函数对比（主瓣宽度与旁瓣衰减）

窗	定义（n = 0…N−1）	主瓣宽度	旁瓣峰值	适用
矩形	w[n] = 1	4π/N（最窄）	−13 dB	要求最高频率分辨率的瞬态分析
汉宁 Hann	0.5 − 0.5cos(2πn/(N−1))	8π/N	−31 dB	通用频谱分析
汉明 Hamming	0.54 − 0.46cos(2πn/(N−1))	8π/N	−43 dB	语音特征提取（最常用）
布莱克曼	0.42 − 0.5cos + 0.08cos(2·)	12π/N	−58 dB	需要极低泄漏的场合

语音和声纹几乎清一色用汉明窗：它在"旁瓣够低"和"主瓣不太宽"之间取得平衡。项目实施里你会在 `wav.py` 里看到这行：

```
w = 0.54 - 0.46*np.cos(2*np.pi*np.arange(400)/(400-1))   # 400 点汉明窗
frame = x[i*160 : i*160+400] * w                          # 取一帧并加窗
```

⑤ 把三步串起来

```
def frame_signal(x, frame_len, hop, win=True):
    n = 1 + (len(x) - frame_len) // hop
    idx = np.arange(frame_len)[None, :] + hop*np.arange(n)[:, None]
    frames = x[idx]                                    # (n, frame_len)
    if win:
        frames = frames * np.hamming(frame_len)        # 逐帧加窗
    return frames

F = frame_signal(x, 400, 160)     # 16 kHz 下 25ms/10ms
print(F.shape)                    # 1 秒 → (98, 400)
```

注意这里用"索引矩阵"一次性取出所有帧，比 for 循环快一个数量级，而且天然支持批量运算——后面做 FFT、做 CNN 输入都靠它。

误区与注意事项

误区 ① "采样率越高越好"。超过信号最高频率两倍的部分不带来新信息，只带来存储与算力开销。② "加窗会丢信息"。加窗是抑制截断造成的高频泄漏，不是丢信息；真正丢信息的是一次只看一小段。③ "帧长越短越好"。帧太短 → 频率分辨率差（后面 MFCC 就分不开相邻谐波）；帧太长 → 违背短时平稳假设。25 ms 是无数实验折中出来的经验值。④ 忘记 **int16** 的 **32767** 上限，导致削波。⑤ 把奈奎斯特上限当可用带宽。8 kHz 的 4000 Hz 是理论极限，实际留 10%–20% 给抗混叠过渡带，用到 3.4 kHz 就算满了。

自检

- [ ] 我能写出 Fs = 800 Hz 采 1000 Hz 正弦时的混叠频率（200 Hz）并解释原因。
- [ ] 我能用 6 dB/bit 算出 16 bit 约 98 dB 动态范围。
- [ ] 我能算出 16 kHz / 25 ms 帧长 = 400 点、10 ms 帧移 = 160 点、1 秒 = 98 帧。
- [ ] 我能解释汉明窗比矩形窗好在哪里、代价是什么。

第 3 讲

时域特征：短时能量、过零率、自相关

目标

- 能手算短时能量、短时过零率、短时自相关三个特征。
- 能用 `ZCR = 2f/Fs` 从过零率反推频率。
- 知道为什么时域特征只能做"快筛"，做不了精细分类。

产出

- 每帧三个特征值，画出一条特征随时间的曲线。
- 能用能量曲线定位"短路冲击"发生在第几帧。

先修知识与准备

需要第 2 讲的分帧（帧长、帧移、加窗）。时域特征不需要傅里叶变换，是整条链路里最容易上手的一步，但它能解决不少实际问题，值得先做。

① 短时能量：把一帧的"响度"变成一个数

物理上，能量与振幅的平方成正比（电阻上的功率 $P = U^2/R$）。对离散信号，一帧内所有采样点平方求和，就得到这一帧的能量：

> $E_m = \Sigma_{n=0}^{N-1} x_m[n]^2$ （第 m 帧，N 为帧长）
>
> 工程上常取对数：$E_m\text{^log} = \log(1 + E_m / a)$，a 是防止 log(0) 的小常数

手算：一帧 4 个采样点 `[1, -2, 0, 2]`，则 $E = 1^2 + (-2)^2 + 0^2 + 2^2 = \mathbf{9}$。若幅值整体放大 2 倍变成 `[2, -4, 0, 4]`，能量变成 **36**——放大 4 倍，因为能量是幅值的平方。这条性质决定了一个重要事实：能量对幅值极敏感，对增益（音量）不稳定。

> **为什么声纹里很少直接用电能作为主特征** 因为录音设备增益一变，绝对能量就全变了。所以能量更多用来做"端点检测"（找出哪里有人说话/有冲击）和"加权"（能量大的帧更可信），而不是直接做分类特征。

② 短时过零率：最便宜的"频率计"

过零率统计一帧内信号穿过零点的次数，反映"变化有多快"，也就是频率高低：

> $Z_m = (1/2) \cdot \Sigma_{n=1}^{N-1} |\text{sgn}(x_m[n]) - \text{sgn}(x_m[n-1])|$
>
> sgn 是符号函数：正数取 1，负数取 −1，0 取 0。除以 2 是因为一个完整周期要穿零两次

推导 **ZCR** 与频率的关系：一个频率 f 的正弦，每秒穿零 2f 次。帧长 N 个点对应时长 N/Fs 秒，所以一帧内的穿零次数为

> $Z = 2f \cdot (N / Fs) \quad \rightarrow \quad f = Z \cdot Fs / (2N)$
>
> 例：Fs = 8000、N = 200（25 ms）、信号 500 Hz
>
> $Z = 2\times500\times200/8000 = 25$ 次 → 反推 $f = 25\times8000/(2\times200) = 500$ Hz √
>
> 另一个说法更常用：过零率 Z/N = 2f/Fs，即"归一化过零率"直接正比于 f/Fs

这个公式在实践中的用法是判清音/浊音：浊音（声带振动，有基频）能量大、过零率低；清音（气流摩擦，像"s"）能量小、过零率高。放到变压器上：风机轴承的宽带噪声、局部放电的高频脉冲，过零率都明显高于 100 Hz 基频主导的正常噪声。

> **注意** 过零率对直流偏置和低频噪声极其敏感：如果信号整体偏了 0.01，本来穿零的地方就不穿了，ZCR 会骤降。所以算 ZCR 前要先去均值（每帧减掉均值），这一步漏掉是最常见的实现错误。

③ 短时自相关：从"重复性"里找周期

自相关衡量信号与"自己平移 k 个点"之后的相似程度：

> $$R_m(k) = \Sigma_{n=0}^{N-1-k} x_m[n] \cdot x_m[n+k]，\ k = 0, 1, ..., K$$

直觉：如果信号每隔 T 个点重复一次，那么把信号平移 T 之后，波形几乎重合，R(T) 就会很大。所以 **R(k)** 的第二个峰（第一个峰在 **k=0**）出现的位置 **k_0**，就是周期：

> 基频估计：$f_0 = Fs / k_0$
>
> 例：Fs = 8000，自相关在 $k_0 = 80$ 处出现峰 → $f_0 = 8000/80 = 100$ Hz √（正是变压器基频）

手算：序列 `[1, 0, -1, 0, 1, 0, -1, 0]`（周期为 4 的方波状信号）。R(0) = 1+0+1+0+1+0+1+0 = 4；R(4) = 1·1 + 0·0 + (−1)(−1) + 0·0 = 2，是 k>0 的最大值 → 周期 4 个点。若 Fs = 8000，则 f_0 = 2000 Hz。

自相关的价值在于：它比 FFT 更"抗噪"地找周期，而且计算量小，所以在基频估计、声门周期检测里用得很多。缺点是对非周期成分（噪声、瞬态）不敏感——这也正好是它的定位。

三个特征的分工

表 3-1　三个时域特征的分工

特征	抓什么	对什么故障敏感	弱点
短时能量	响度/幅度	短路冲击（能量突增）、风机老化（整体能量抬升）	受增益影响，不能区分同能量的不同故障

特征	抓什么	对什么故障敏感	弱点
短时过零率	变化快慢 ≈ 频率高低	局部放电（高频脉冲）、风机（宽带噪声）	对直流偏置敏感，高频噪声下会饱和
短时自相关	周期性与基频	铁芯松动（周期结构改变）、直流偏磁（倍频成分）	对非周期瞬态不敏感

④ 为什么只有时域特征不够

设想两种故障：一个是"100 Hz 能量增大"，一个是"200 Hz 能量增大"，但两者总能量相同。它们的短时能量完全一样，过零率也很接近（100 与 200 Hz 都远低于 8 kHz 的一半），自相关只能给出周期变化但很难稳定区分——时域特征把频率信息压成了一个标量，信息在这里就丢了。

这就是必须进入频域的原因：要区分"能量分布在哪些频率上"，就必须把信号拆成一个个频率成分——这正是第 4 讲傅里叶变换要干的事。工程上的惯例是：时域特征做快速筛查和端点检测，频域/时频域特征做精细分类。

⑤ 代码：一次算出三条曲线

```
def frame_feats(x, N=400, S=160):
    n = 1 + (len(x) - N)//S
    idx = np.arange(N)[None,:] + S*np.arange(n)[:,None]
    F = x[idx] - x[idx].mean(1, keepdims=True)      # 去均值（对 ZCR 必须）
    E = (F**2).sum(1)                                # 短时能量
    Z = 0.5*np.abs(np.diff(np.sign(F), axis=1)).sum(1)   # 过零率
    R = np.array([np.correlate(f, f, 'full')[N-1:] for f in F])  # 自相关
    return E, Z, R

E, Z, R = frame_feats(x)
print('能量最大帧', E.argmax(), '峰值', round(E.max(), 1))
```

对"短路冲击"样本跑一遍，你会看到能量曲线在某一帧突然跳起 1~2 个数量级——只看这一条曲线就能定位冲击发生在 **0.5** 秒里的第几毫秒。这就是时域特征最实用的场景。

误区与注意事项

误区 ① 算 **ZCR** 不去均值。直流偏置会让过零率完全失真。② 把能量当"音色"特征。能量是响度，换个录音增益就变。③ 用自相关找周期却不限制 **k** 范围。k 应该限制在"可能的基频范围"内（如 50–500 Hz 对应 k = 16–160 @8 kHz），否则会被倍周期或噪声峰骗到。④ 拿一帧的结果当结论。单帧特征抖动很大，实际都按"一段时间的统计量"（均值、方差、分位数）使用。

自检

- [ ] 我能手算 `[1,-2,0,2]` 的短时能量（9）。
- [ ] 我能用 `Z = 2fN/Fs` 算出 500 Hz、200 点帧、8 kHz 下的过零率（25 次）。
- [ ] 我能解释自相关峰位置怎么换算成基频。
- [ ] 我能说出时域特征为什么分不开"同能量的不同频率故障"。

第 4 讲

傅里叶变换：从时间到频率

目标

- 把 DFT 理解成"在一组正弦上的投影"，而不是一串看不懂的指数。
- 能手算 4 点 DFT，能算频率分辨率 Δf。
- 理解补零、频谱泄漏、单边谱归一化三个工程细节。

产出

- 手写 DFT，与 `numpy.fft` 对拍误差小于 1e−10。
- 画出复合音频谱，能指着峰说出"这是 100 Hz 基频，这是 2、3 次谐波"。

配套可拖演示 打开同目录下的 `labs.html`（第一个标签页），把探测频率从 0 拖到 4000：当 f 命中真实分量时，圆里的点会朝一边堆叠、质心明显偏离圆心；f 对不上时点铺满一圈、质心回到圆心。那段动画就是 DFT 本身，比任何文字解释都直接。

先修知识与准备

需要：复数与欧拉公式 `e^(jθ) = cosθ + j·sinθ`、内积（点乘）、求和符号 Σ。这三样就够，不需要任何泛函知识。

① 核心直觉：频谱是"投影系数"

先看一个平面几何事实：向量 `a=(1,0)` 与 `b=(0,1)` 的内积为 0，叫正交。正交的好处是"互不干扰"：你想知道 a 在 x 方向的分量，就用 a 和 x 轴单位向量做内积，y 方向的存在完全不影响结果。

傅里叶变换做的正是这件事，只不过向量换成了"函数"，内积换成了"相乘再求和/积分"。频率为 f 的复指数 `e^(j2πft)` 就是"频率轴上的单位向量"。不同频率的复指数彼此正交：

> $\int e^{j2\pi f_1 t} \cdot \mathrm{conj}(e^{j2\pi f_2 t})\, dt = 0$ （当 $f_1 \neq f_2$）
>
> 所以"信号在频率 f 上的分量" = 信号与 $e^{j2\pi ft}$ 的内积 = 信号与那个频率的相似程度

这就是为什么频谱能"分离混在一起的谐波"：各频率互不干扰，各投各的影。理解了这一句，DFT 的公式就不需要背了。

② DFT 公式与每个符号的含义

$X[k] = \Sigma_{n=0}^{N-1} x[n] \cdot e^{-j \cdot 2\pi \cdot k \cdot n/N}$，　$k = 0, 1, \ldots, N-1$

$x[n]$：第 n 个采样点（时域）

$X[k]$：第 k 个频率分量（频域，复数）

$e^{-j2\pi kn/N}$：第 k 个"基函数"，就是一个频率为 k 的复正弦

k 对应的物理频率：$f_k = k \cdot Fs / N$

逐项翻译：对每个 k，把整段信号 x[n] 与第 k 个复正弦逐点相乘再求和。如果信号里含有这个频率，乘积同号居多，求和结果就大；如果信号里没有，正负相消，结果接近 0。`X[k]` 是复数，模长是"这个频率有多强"，辐角是"这个频率的相位"。

记住一个换算 频率分辨率 **$\Delta f = Fs / N$**，即第 k 根谱线对应 $k \cdot \Delta f$。这个数字决定了"两个频率能不能分开"。例：Fs = 8000、N = 256 → Δf = 31.25 Hz，意味着 100 Hz 和 120 Hz 两个成分在图上会糊在一起；N = 1024 → Δf = 7.8 Hz，就能分开。

③ 手算一个 4 点 DFT

取 `x = [1, 0, 1, 0]`（周期为 2 的交替序列），N = 4。基函数是 4 个点上的复指数，取值只有 `1, -j, -1, j` 四种：

k=0：$X[0] = 1 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 = 2$

k=1：$X[1] = 1 \cdot 1 + 0 \cdot (-j) + 1 \cdot (-1) + 0 \cdot (j) = 1 - 1 = 0$

k=2：$X[2] = 1 \cdot 1 + 0 \cdot (-1) + 1 \cdot 1 + 0 \cdot (-1) = 2$

k=3：$X[3] = 1 \cdot 1 + 0 \cdot (j) + 1 \cdot (-1) + 0 \cdot (-j) = 0$

结果：X = [2, 0, 2, 0]

怎么读这个结果？X[0]=2 是直流分量（均值×N = 0.5×4 = 2 √）；X[2]=2 对应频率 k=2，即 $f = 2 \cdot Fs/4 = Fs/2$，正是这个"1,0,1,0"序列的交替频率；X[1]=X[3]=0 说明没有其他成分。算一遍就明白：**DFT** 只是把信号往每个频率上射一次影。

④ 三个必须搞清的工程细节

细节 1：单边谱要乘 2/N 才是真实幅值

实信号的频谱是共轭对称的（$X[N-k] = \text{conj}(X[k])$），负频率没有独立信息，所以工程上只看前 N/2+1 根谱线。此时那根谱线的模要除以 N/2 才等于正弦的真实振幅：

单边幅度谱：$A[k] = 2 \cdot |X[k]| / N$　（k = 0 除外，直流不乘 2）

例：幅值 1.0 的 100 Hz 正弦，N = 1024，在 k 对应 100 Hz 处 $|X[k]| \approx 512 \rightarrow A = 2 \times 512/1024 = 1.0$ √

忘了这个 2/N，画出来的谱幅值是 512 而不是 1，就会得出"信号幅值 512 Pa"这种荒谬结论——这是初学者最常见的错误之一。

细节 2：补零不能提高分辨率

把 256 点信号补零到 1024 点再做 FFT，谱线看起来"更密"（Δf 从 31.25 变成 7.8 Hz），但分辨率并没有变好。因为分辨率由"实际观测时长 T = N_original/Fs"决定，补零只是把同一条曲线插值画得更细——本来糊在一起的两个峰，补零后还是糊在一起，只是多画了几个点。

> 真实分辨率 $\Delta f = 1 / T_{obs} = Fs / N_original$
>
> 补零到 M 点只改变"画图密度"：$\Delta f_display = Fs / M$，不改变分辨能力

要用好这条：需要分辨间隔 5 Hz 的两个成分，观测时长至少 1/5 = 0.2 s。这也是为什么声纹分析一般用 0.2–1 s 的片段而不是 25 ms 单帧。

细节 3：频谱泄漏

如果信号频率不是 Δf 的整数倍（绝大多数情况都是），能量会"漏"到相邻谱线上，出现一个主峰加两边一串小峰。第 2 讲的加窗就是为了压这个泄漏。判断方法：看一个纯正弦的谱，如果除了主峰还有明显的旁瓣，就是泄漏。

⑤ FFT：为什么能快两个数量级

直接按公式算 N 点 DFT，需要约 N^2 次复数乘法。N = 1024 时是 100 万次。Cooley–Tukey 的分治思路是：把 N 点 DFT 拆成两个 N/2 点 DFT（偶数点一组、奇数点一组），再递归下去：

> 复杂度：DFT 约 N^2 次复数乘法 → FFT 约 $(N/2)\cdot\log_2 N$
>
> $N = 1024$：DFT ≈ 1,048,576 次，FFT ≈ 5120 次 —— 相差 200 倍
>
> $N = 8192$：DFT ≈ 6.7×10^7，FFT ≈ 5.3×10^4 —— 相差 1200 倍

这就是为什么所有实时系统都要求 FFT 长度是 2 的幂：不是数学要求，而是分治算法要求能一直二分下去。

⑥ 代码：三行 DFT 与对拍

```
import numpy as np
def dft(x):
    N = len(x); k = np.arange(N); n = k.reshape(-1,1)
    W = np.exp(-2j*np.pi*k*n/N)         # 基矩阵：每一行是一个频率
    return W @ x                          # 一次矩阵乘法 = 一组内积

x = np.sin(2*np.pi*100*np.arange(256)/8000)
X = dft(x)
print('与 numpy 最大偏差', np.abs(X - np.fft.fft(x)).max())   # ~1e-13
idx = np.abs(X[:128]).argmax()
print('峰值频率', idx*8000/256, 'Hz')                         # ≈ 100 Hz
```

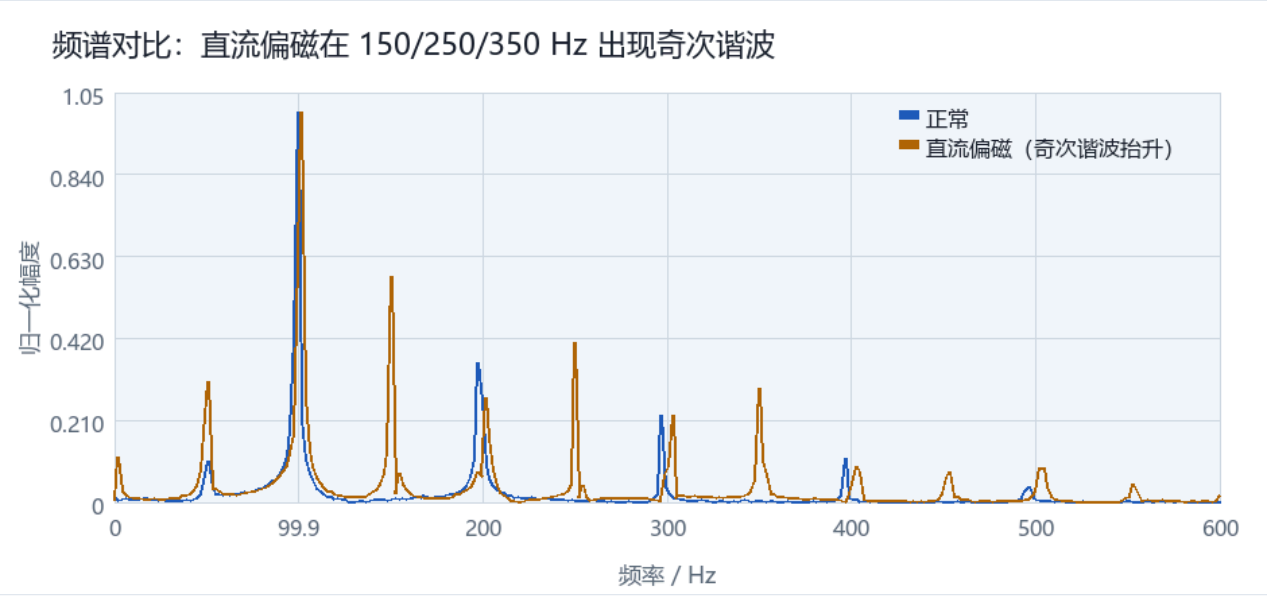


图 4-1　六类声纹的频谱对比：正常以 100 Hz 基频为主，直流偏磁在 150/250/350 Hz（50 Hz 的奇次倍频）出现明显新增峰——不同故障在频域有不同"指纹"

误区与注意事项

误区 ① 补零 = 提高分辨率。错，分辨率由观测时长决定。② 忘记 **2/N** 归一化。③ 把 **X[0]** 当"频率 **0** 的分量"却不知它是直流/均值×**N**。④ 拿负频率当"另一种信号"。实信号的负频率只是共轭镜像，没有新信息。⑤ **FFT** 点数随便取非 **2** 的幂。能用但慢，实时系统会要命。

自检

- [ ] 我能用"投影/内积"解释 DFT 在做什么。
- [ ] 我能手算 `[1,0,1,0]` 的 4 点 DFT 得 `[2,0,2,0]`。
- [ ] 我能算出 8 kHz、256 点 FFT 的 Δf=31.25 Hz，并说出它能分开多近的两个频率。
- [ ] 我能解释"补零不提高分辨率"和"频谱泄漏"两件事。

第 5 讲

时频分析：STFT 与不确定原理

目标

- 理解 FFT 丢了时间信息，STFT 如何把它补回来。
- 能说清"窗长"这个唯一旋钮在时间与频率之间怎么交换。
- 会读语谱图：水平条纹、竖直条纹、颜色深浅各代表什么。

产出

- 一张 log-Mel 语谱图，并能在图上指出局放脉冲出现的时刻。
- 能算出一组窗长对应的 Δf，并说明为什么选它。

先修知识与准备

需要第 2 讲的分帧加窗、第 4 讲的 DFT 与 $\Delta f = Fs/N$。本讲是把这两件事拼起来，没有新数学。

① FFT 的盲区：它只知道"有哪些频率"，不知道"什么时候"

对整段 0.5 秒信号做一次 FFT，得到的是这段时间的平均频谱。如果局放脉冲只出现在第 200 ms，FFT 会告诉你"存在高频成分"，但完全无法告诉你在第 200 ms。对于稳态故障（铁芯松动、风机老化）这没关系，但瞬态故障（局放、短路冲击）的时间位置本身就是最重要的判别信息。

STFT 的思路很直接：把长信号切成很多短帧，每帧做一次 **FFT**，再把结果按时间排起来。这样每一列是一个时刻的频谱，横着看就是频率随时间的变化。

> STFT：$X[m, k] = \Sigma_n\, x[n] \cdot w[n - m \cdot S] \cdot e^{-j \cdot 2\pi \cdot k \cdot n/N}$
>
> m：第几帧（时间轴）；k：第几根谱线（频率轴）
>
> w：窗函数；S：帧移
>
> 输出是一个二维复数矩阵，尺寸 = 帧数 × (N/2+1)

② 唯一要调的旋钮：窗长

窗长决定了两件事，而且是矛盾的：

> 窗长 L → 时间分辨率 $\Delta t \approx L / Fs$
>
> 窗长 L → 频率分辨率 $\Delta f = Fs / L$
>
> 两者乘积 $\Delta t \cdot \Delta f \approx 1$ （这是不确定原理的工程形态，不可能同时变小）

带数字算两遍（8 kHz 采样）：

表 5-1　不同窗长对应的时间与频率分辨率（8 kHz）

窗长	点数 L	时间分辨率 Δt	频率分辨率 Δf	适合看
5 ms	40	5 ms	200 Hz	瞬态冲击、局放脉冲
25 ms	200	25 ms	40 Hz	谐波结构（最常用）
100 ms	800	100 ms	10 Hz	精细谐波、低频率成分

看懂这张表，就理解了 STFT 的全部权衡：窗越短，时间越清楚但频率越模糊；窗越长，频率越清楚但时间越模糊。这不是算法不够好，是物理限制（韩书用柯西–施瓦茨不等式证明过，等号只在 Gabor 变换——高斯窗——时成立）。

③ 为什么必须取对数显示

语音/声纹的能量动态范围很大：基频能量可能比第 10 次谐波高 40 dB（1 万倍）。如果按线性幅度画图，弱谐波在图上就是一片全黑，什么也看不见。取对数后：

> 显示值 = $20 \cdot \log_{10}(|X[m,k]| + \varepsilon)$ （dB）
>
> 40 dB 的差距在图上变成"一格半"的灰度差，弱结构就看得见了

这就是为什么所有语谱图都标着 dB，也是本讲义里所有谱图都用 `log` 显示的原因。

④ 怎么读一张语谱图

下面这张是"局部放电"样本的 log-Mel 语谱图。学会读它，比记公式更有用：

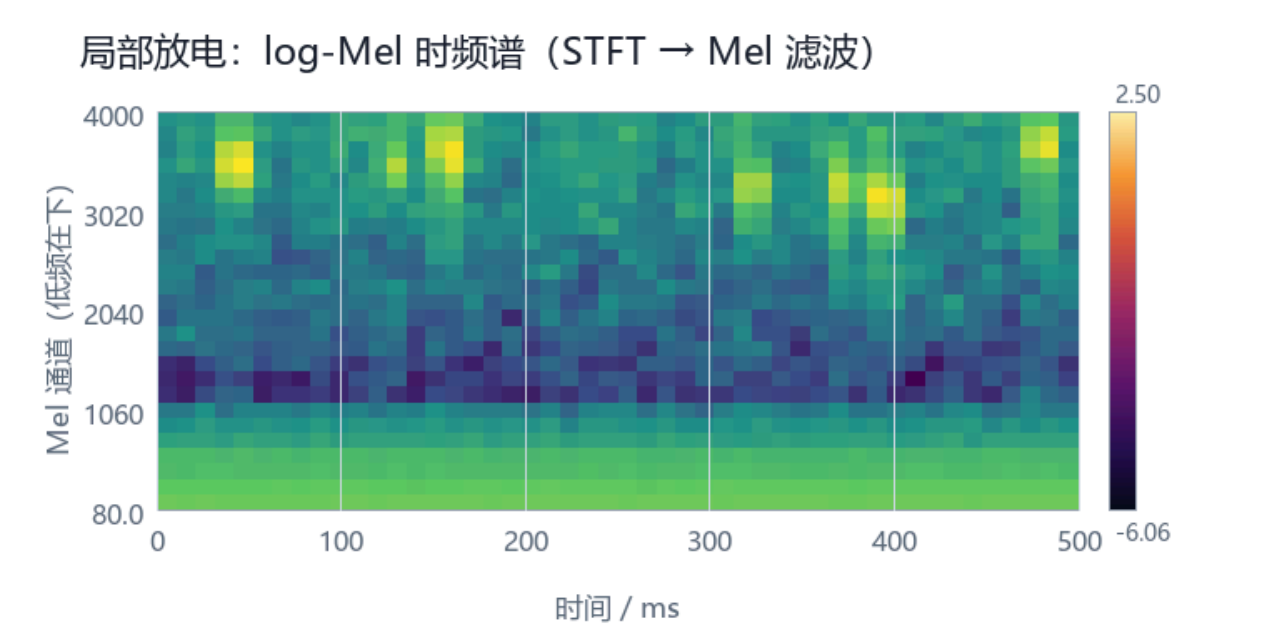


图 5-1　局放样本的 log-Mel 时频图：横轴时间、纵轴 Mel 通道（近似频率）、颜色是能量（dB）。竖直的亮条就是放电脉冲出现的时刻；底部连续的亮带是 100 Hz 基频及其谐波

表 5-2　语谱图常见形态与对应故障

你在图上看到	物理含义	典型故障
底部一条水平亮线	基频（100 Hz）持续存在	正常运行也有
等间隔的多条水平线	谐波族（200/300/400 Hz…）	谐波比例变化 → 铁芯松动、偏磁
竖直亮条（贯穿多个频率）	极短的宽频瞬态	局部放电、短路冲击
整片均匀变亮（高频段）	宽带噪声抬升	风机轴承磨损
低频段出现新的间隔峰	50 Hz 倍频族	直流偏磁

⑤ 代码：从波形到语谱图

```
def stft(x, N=200, S=80, win=True):
    n = 1 + (len(x)-N)//S
    idx = np.arange(N)[None,:] + S*np.arange(n)[:,None]
    F = x[idx]
    if win: F = F * np.hamming(N)
    return np.fft.rfft(F, axis=1)          # (帧数, N/2+1) 复数

X = stft(x)                                 # 8 kHz、25 ms 窗、10 ms 移
S_db = 20*np.log10(np.abs(X) + 1e-8)        # 转 dB
print(S_db.shape)                           # (帧数, 101)
```

注意 `rfft`：实信号只算正频率那一半，输出 101 列（200/2+1），比 `fft` 省一半内存。项目里还多做了一步 Mel 滤波（第 6 讲），把 101 个频率通道压到 26 个，这样图更"干净"、维度更低。

误区与注意事项

误区 ① 用一个窗长应付所有故障。看瞬态用短窗、看谐波用长窗，选错了会"看不见"关键信息。② 用线性幅度画语谱图。弱谐波全部淹没。③ 以为 **STFT** 能突破不确定原理。任何改进（小波、EMD）都只是"按需分配分辨率"，不是突破。④ 帧移取得比窗长还大。会漏掉帧与帧之间的信号，瞬态可能整个被跳过。

自检

- [ ] 我能说出 STFT 相比 FFT 多给出了什么信息。
- [ ] 我能算出 8 kHz 下 5/25/100 ms 窗对应的 Δf（200/40/10 Hz）。
- [ ] 我能解释为什么语谱图要取 dB。
- [ ] 我能在语谱图上区分"水平谐波条"和"竖直瞬态条"，并说出各对应什么故障。

第 6 讲

MFCC：最常用的声学特征

目标

- 能完整说出 MFCC 的 7 个步骤，并解释每一步"模拟了人耳的什么"。
- 能算出 Mel 滤波器的中心频率，理解为什么低频密、高频疏。
- 知道为什么最后要用 DCT、为什么加一阶二阶差分才是关键。

产出

- 一段 MFCC 代码，输出 (帧数, 13) 与 (帧数, 39)。
- 一张 Mel 滤波器组图，能指着说"这是第 3 个滤波器的三角"。

先修知识与准备

需要第 4 讲的 FFT 与功率谱、第 5 讲的加窗与帧参数、以及对数运算。DCT 会在本节内讲清，不需要提前学。

① 为什么不能直接拿频谱当特征

第 5 讲的语谱图其实已经能用了——把 101×48 的矩阵直接喂给 CNN 就是"频谱图 + CNN"的方案（第 9 讲会做）。但作为向量特征它有三个问题：维度高（每帧 101 维）、相邻频率高度相关（冗余）、对人耳不敏感的高频区给了过多权重。

MFCC 就是为了解决这三点，用一条"人耳仿真流水线"把一帧 400 个采样点压成 13 个数。下面是完整 7 步，每一步都标注它在模仿人耳的哪个环节。

② 第 1 步：预加重

> $y[n] = x[n] - \alpha \cdot x[n-1]$，　α 通常取 0.95 ~ 0.97
>
> 作用：提升高频（相当于 6 dB/倍频程的高通），抵消口唇辐射造成的高频衰减

为什么需要它？声带产生的声音经过口腔辐射出去时，高频能量会以约 6 dB/倍频程衰减。预加重把这个倾斜"抬平"，让 FFT 之后的频谱更好分析、动态范围更小。

> **注意（这条对变压器声纹很关键）** 预加重是为语音设计的。变压器声纹的主要判别信息在 100 Hz 低次谐波上，预加重会把低频相对压低——所以做设备声纹时，很多工作会跳过预加重或改用很小的 α。这是"语音方法迁移到设备声纹"时最典型的坑之一。

③ 第 2–3 步：分帧、加窗、FFT 取功率谱

> 帧长 25 ms、帧移 10 ms、汉明窗（第 2 讲）
>
> 每帧做 N 点 FFT（N 取 256 或 512，补零到 2 的幂）
>
> 功率谱：$P[k] = |X[k]|^2 / N$　　（$k = 0 \dots N/2$）

以 8 kHz、25 ms（200 点）为例，补零到 256 点做 FFT，得到 129 根谱线，频率间隔 8000/256 = 31.25 Hz。

④ 第 4 步：Mel 滤波器组——把人耳的"非线性"装进来

人耳对频率的感知不是线性的：100 Hz 与 200 Hz 的差别听起来很大，而 3000 Hz 与 3100 Hz 几乎听不出。为了让特征符合听觉，用 Mel 刻度重排频率轴：

> $\mathrm{mel}(f) = 2595 \cdot \log_{10}(1 + f / 700)$
>
> 反变换：$f = 700 \cdot (10^{\mathrm{mel}/2595} - 1)$
>
> 例：$f = 0 \to 0$ mel；$f = 1000$ Hz $\to 2595 \cdot \log_{10}(2.43) \approx 1000$ mel（有趣的巧合）
>
> $f = 4000$ Hz $\to 2595 \cdot \log_{10}(6.71) \approx 2146$ mel

做法：在 Mel 轴上等间隔放 M 个三角滤波器（通常 M = 24~26），每个滤波器的中心在 Mel 轴上等距，覆盖 0 到 Fs/2。因为 Mel 轴是压缩过的，映射回 Hz 之后，低频的滤波器又窄又密，高频的又宽又疏——这正是人耳的频率分辨特性。

8 kHz 采样、26 个滤波器时的中心频率（前 10 个）

表 6-1　8 kHz、26 个 Mel 滤波器的中心频率与带宽（前 10 个）

序号	1	2	3	4	5	6	7	8	9	10
中心频率 / Hz	51	106	165	228	296	368	445	528	616	711
带宽 / Hz	51	55	59	63	68	72	77	83	88	95

看带宽这一行：从 51 Hz 一路涨到 95 Hz，后面的滤波器的带宽会到 300 Hz 以上。所以 MFCC 在 100–500 Hz 这一段的分辨率最高——刚好覆盖了变压器声纹最关心的基频与低次谐波。

每个滤波器的输出是该频带内的加权能量和：

> $E_m = \Sigma_k P[k] \cdot H_m[k]$　　（H_m 是第 m 个三角滤波器的频率响应）
>
> 这一步把 129 根谱线压成 26 个数，同时完成了"降维 + 按听觉加权"

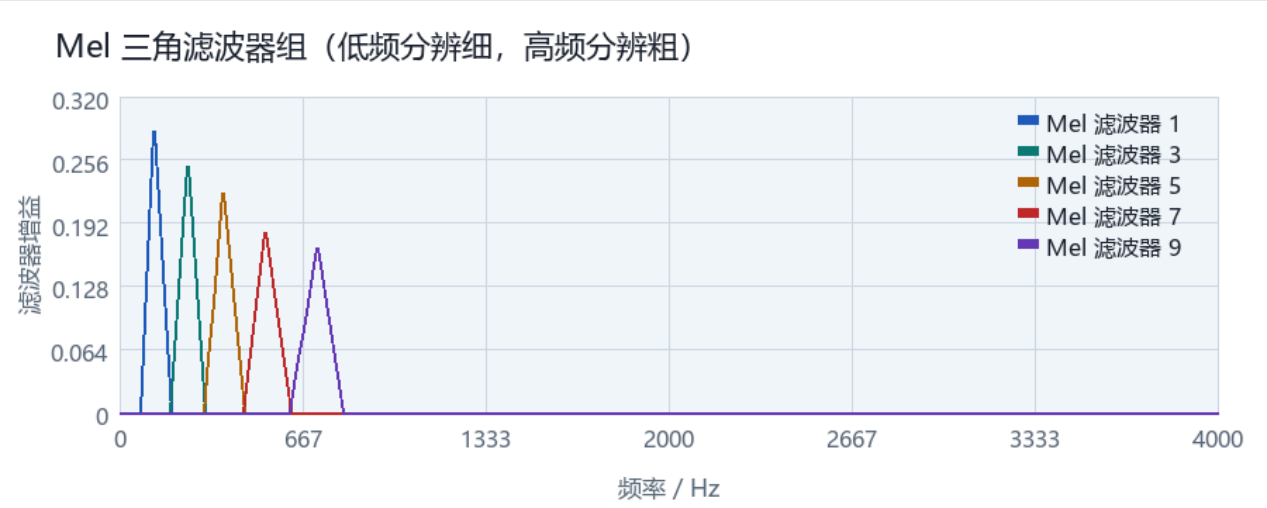


图 6-1　26 个三角滤波器在频率轴上的排布：低频尖而密，高频宽而疏，这就是 Mel 刻度的形状

⑤ 第 5 步：取对数

$$L_m = \log(E_m + \varepsilon)$$

两个理由。物理上：人耳对响度的感知近似对数（这也是 dB 存在的原因）。工程上：声源与信道在时域是卷积（x 经过信道 h 变成 x*h），在频域是相乘，取对数后变成相加——于是信道的影响变成一个加性常量，后面的倒谱均值归一化（CMN）能直接把它减掉。这一步是"倒谱"这个名字的由来（cepstrum = spectrum 的前四个字母倒过来）。

⑥ 第 6 步：DCT，把 26 维压成 13 维

相邻的 Mel 通道高度相关（第 3 个和第 4 个滤波器覆盖的频带是重叠的），需要去相关。DCT（离散余弦变换）就是为此设计的正交变换：

$$c_i = \Sigma_{m=1}^{M} L_m \cdot \cos[\,(\pi \cdot i/M) \cdot (m - 0.5)\,], \quad i = 0, 1, \ldots, M-1$$

c_0 反映总能量（常丢弃或单独用），$c_1 \ldots c_{12}$ 反映频谱包络的形状

通常只保留前 13 个系数（含 c_0）或 $c_1 \ldots c_{13}$

为什么 DCT 有效？因为 Mel 对数谱是一条平滑的曲线，能量集中在"低频"（即系数下标小的那几个）——前几个系数就描述了整条包络的形状，后面的系数对应快速起伏的细节（多为噪声）。丢掉后面的系数，既降维又降噪。

注意：DCT 与 PCA 目的相同（去相关+能量集中），但 DCT 的基是固定的余弦函数，不需要从数据里学，所以计算极快、也不存在"换数据集就要重算"的问题。

⑦ 第 7 步：加一阶、二阶差分（这一步比加维度重要得多）

静态 MFCC 描述的是"这一帧的频谱形状"，但声音是动态的：辅音到元音的过渡、敲击的起始沿，信息都在变化里。差分特征就是帧间的斜率：

$$d_t = \Sigma_{n=1}^{N} n\cdot(c_{t+n} - c_{t-n}) \,/\, (2\cdot\Sigma_{n=1}^{N} n^2)$$

$N = 2$ 时：$d_t = [(c_{t+1}-c_{t-1}) + 2(c_{t+2}-c_{t-2})] \,/\, 10$

二阶差分：对 d_t 再做一次同样的运算

维度变化：13（静态）+ 13（一阶 Δ）+ 13（二阶 ΔΔ）= **39** 维。

一条被反复验证的实测结论 韩书表 3-3 的实验结果是：单纯增加倒谱维数（13 → 20 → 30）几乎不降低误识率；而加入一阶、二阶差分能让误识率下降约 20%。原因就是上面说的：静态特征丢掉了时间演化信息。所以做声纹一定要加 Δ/ΔΔ，别只堆维度。

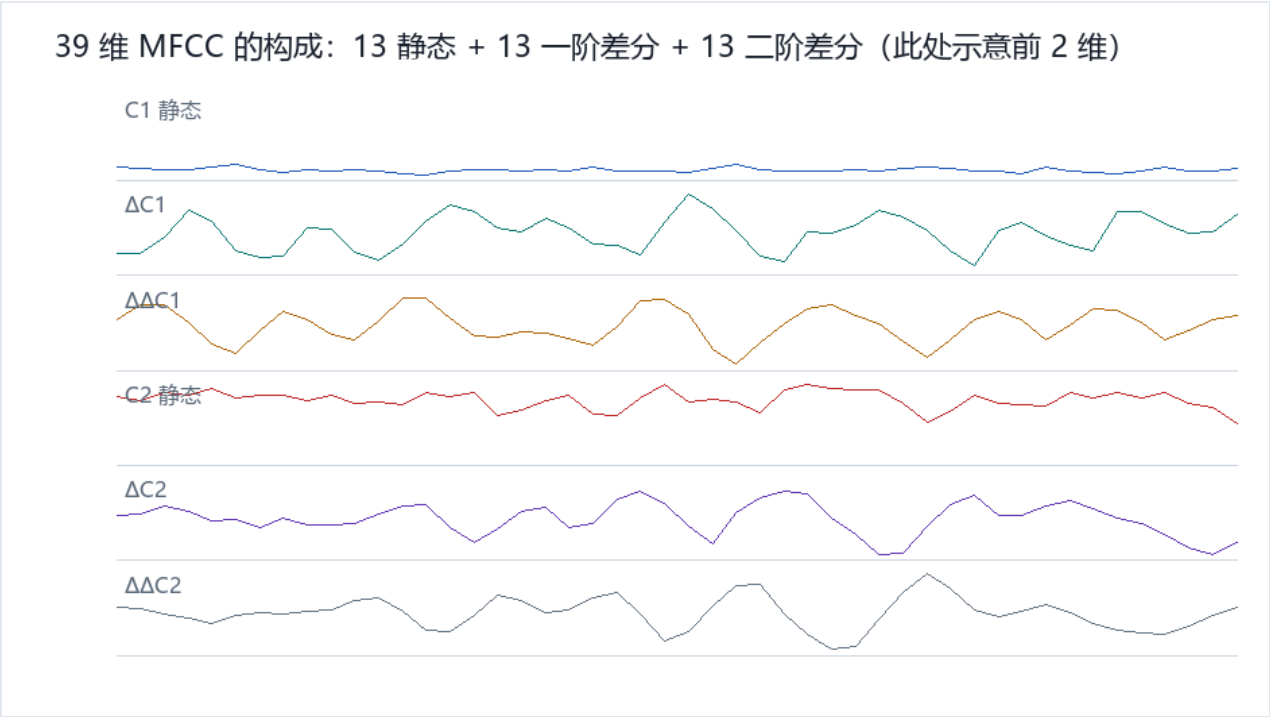


图 6-2　39 维 MFCC 的构成：C1 静态、ΔC1 一阶差分、ΔΔC1 二阶差分（此处示前 2 维）。差分曲线明显比静态曲线"更能看出节奏"

⑧ 维度账与标准化

```
def mfcc(x, fs, n_mels=26, n_ceps=13, N=256, frame=200, hop=80):
    F = frame_signal(x, frame, hop)                 # (T, 200)
    P = np.abs(np.fft.rfft(F, N, axis=1))**2 / N    # (T, 129)
    fb = mel_filterbank(n_mels, N, fs)              # (26, 129)
    E = P @ fb.T + 1e-10                            # (T, 26)
    C = dct(np.log(E))[:, :n_ceps]                  # (T, 13)
    return C

C  = mfcc(x, 8000)                # (T, 13)
D  = delta(C); DD = delta(D)
feat = np.hstack([C, D, DD])      # (T, 39)
```

算一下压缩比：0.5 秒、8 kHz = 4000 个采样点；25 ms/10 ms 分帧 → 23 帧；23×39 = **897** 个数。压缩到原来的 1/4.5，而判别信息基本保留。

最后一步是倒谱均值归一化（**CMN**）：每个系数减去它在整段语音上的均值。这一步专门对付信道差异（不同麦克风、不同距离），是声纹系统必备的后处理。

误区与注意事项

误区 ① 照搬语音的预加重到设备声纹。会把 100 Hz 基频相对压低，等于扔掉主要判别信息。② 只取 **13** 维静态 **MFCC** 就下结论。不加 Δ/ΔΔ 会白白多出约 20% 的误识。③ 以为 **Mel** 滤波器越多越好。太多会引入冗余和过拟合，26 是语音的经验值；设备声纹常取 24–40，需实验。④ 忘了 **CMN**。换一套录音设备就崩。⑤ 把 **c_0** 当特征用却不理解它。c_0 近似总能量，对增益敏感。

自检

- [ ] 我能按顺序说出 MFCC 七步，并说出每步模仿人耳的哪一部分。
- [ ] 我能用 $mel(f)=2595\cdot\log_{10}(1+f/700)$ 算出 1000 Hz 约等于 1000 mel。
- [ ] 我能解释为什么低频滤波器密、高频疏。
- [ ] 我知道 13→39 维是怎么来的，也能说出加差分比加维度更有用。

第 7 讲

小波变换与 EMD：超越固定窗的时频分析

目标

- 理解 STFT 的"固定窗"为什么在宽频带信号上吃亏。
- 能说清连续小波、离散小波、EMD 三种方法各自的适用场景与代价。
- 知道 EMD 的三个已知缺陷，不会盲用它。

产出

- 一张小波尺度图，能在图上定位局放脉冲的时刻。
- 一组 IMF 分量图，能说清每层 IMF 对应哪个时间尺度。

先修知识与准备

需要第 4 讲的频域概念、第 5 讲的时频权衡。小波的数学（多分辨分析）可以先跳过，本讲只讲"它在干什么、什么时候用"。

① STFT 的困境：一个窗长管所有频率

第 5 讲说过，窗长决定时间/频率分辨率的分配。问题在于 STFT 用同一个窗长处理所有频率。但实际信号的需求正好相反：

表 7-1　STFT 在宽频带信号上的两难

你关心的成分	需要的分辨率	STFT 的处境
局放脉冲（微秒级、宽频）	时间要极短	用长窗 → 脉冲被抹平，看不出时刻
100 Hz 基频与相邻谐波（间隔 50 Hz）	频率要极窄	用短窗 → 谐波糊成一片

小波的解法是：用"可伸缩的窗"——分析高频时用很短的窗，分析低频时用很长的窗。这正好符合"高频变化快、需要看细；低频变化慢、可以看粗"的直觉。

② 连续小波变换：把"频率"换成"尺度"

$$W(a, b) = \int x(t) \cdot \psi^*((t - b)/a)\, dt$$

ψ：母小波（mother wavelet），一个"小段振荡、两端衰减到 0"的函数

a：尺度（scale）→ 控制伸缩，a 大 = 窗宽 = 看低频

b：平移（translation）→ 控制位置，b 就是时间

尺度与频率的换算关系是理解小波图的关键：

> $f \approx f_c \cdot Fs / a$ （f_c 是母小波的中心频率，Morlet 小波约 0.8~1）
>
> 例：$Fs = 8000$，$a = 8 \rightarrow f \approx 1000$ Hz
>
> $Fs = 8000$，$a = 80 \rightarrow f \approx 100$ Hz

常用的母小波是复 Morlet：一个高斯包络调制的复正弦。用复数小波的好处是能同时得到幅值和相位，幅值图就是"该尺度在该时刻有多强"。

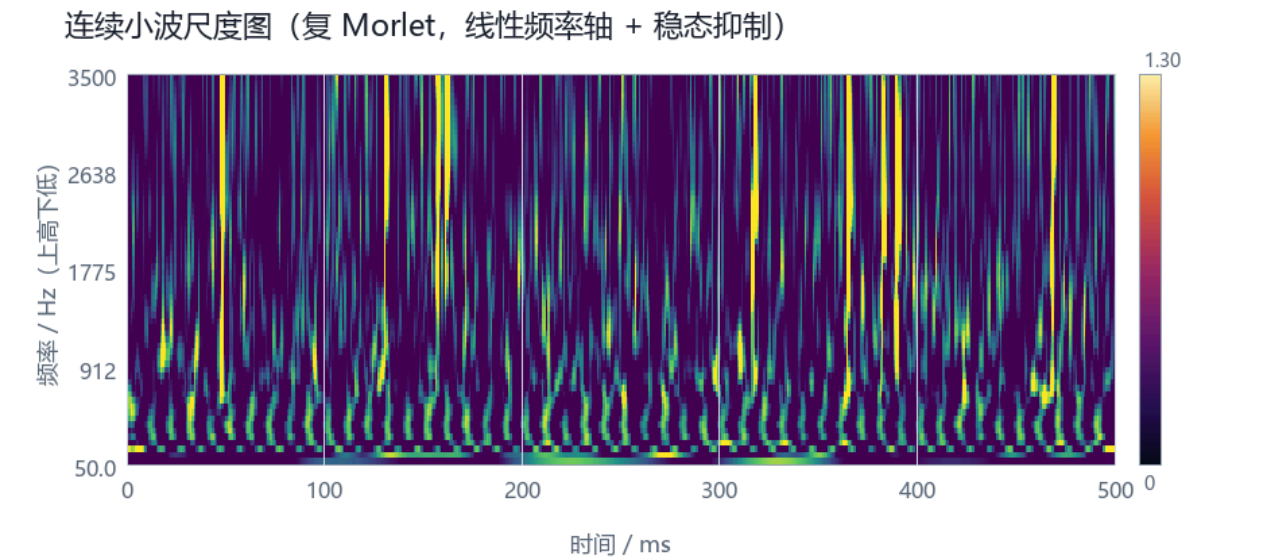


图 7-1　局放样本的 Morlet 尺度图（已换成线性频率轴并做稳态抑制）：竖直亮条是放电脉冲出现的时刻，底部连续亮带是 100 Hz 基频谐波。这张图是全局 **FFT** 给不出的信息

③ 离散小波变换：工程上真正在用的版本

连续小波要在所有 a、b 上算积分，计算量巨大。工程上用"二进离散"：a 取 2 的幂（1, 2, 4, 8…），b 取整数倍，得到 DWT。它可以用一对滤波器（低通 h_0、高通 h_1）反复迭代实现，这就是 Mallat 算法：

> 第 1 层：$x \rightarrow$ 低通+降采样 $\rightarrow cA_1$（近似）；高通+降采样 $\rightarrow cD_1$（细节）
>
> 第 2 层：$cA_1 \rightarrow cA_2, cD_2$ ……
>
> 3 层分解后：$cA_3 + cD_3 + cD_2 + cD_1$ 可以无损重构回原信号回原信号

算一下长度（N = 4000）：

表 7-2　3 层 DWT 的系数长度与对应频带

层	输出	长度	对应频带（8 kHz）
1	cD_1	2000	2000–4000 Hz
2	cD_2	1000	1000–2000 Hz
3	cD_3	500	500–1000 Hz

层	输出	长度	对应频带（8 kHz）
3	cA_3	500	0–500 Hz

这就是"多分辨分析"：细节系数按 **2** 的幂切分频带，高频用短时间窗、低频用长时间窗，自动完成。它常被用来做特征——各层系数的能量、能量熵、方差，几乎就是"各频带有多活跃"的度量。

```
# 3 层 Haar（最简单的正交小波）分解与重构
def dwt_haar(x):
    c = np.asarray(x, float)
    a = (c[0::2] + c[1::2]) / np.sqrt(2)      # 近似（低通）
    d = (c[0::2] - c[1::2]) / np.sqrt(2)      # 细节（高通）
    return a, d
# 逐层分解后把系数反算回去，重构误差应在 1e-16 量级
```

一个实现上的硬要求 正交小波的重构必须用分析算子的转置，不能"另写一套卷积"。项目里第一次写 db4 重构时误差是 1.4（完全错），原因就是把重构写成了另一套滤波器相位；改成显式使用 A^T 之后误差降到 1e−12。判断标准很简单：重构误差不是 **1e−12** 量级，就一定有错。

④ EMD：连基函数都不要了

小波有个前提问题：母小波要人挑（Haar？db4？Morlet？），挑错了效果就不好。EMD（经验模态分解）的思路是让数据自己决定基函数：把信号拆成若干个"本征模态函数"（IMF），每个 IMF 满足两个条件：

① 极值点个数与过零点个数相差不超过 1

② 上包络（极大值拟合）与下包络（极小值拟合）的均值为 0（局部对称）

筛分（sifting）的流程：

① 找 x 的所有局部极大值、极小值

② 用三次样条插值连成上包络 e_{max} 与下包络 e_{min}

③ 求均值 $m = (e_{max} + e_{min})/2$

④ 令 $h = x - m$，判断 h 是否满足 IMF 条件；不满足就把 h 当新的 x 重复 ①②③

⑤ 满足则记为一个 IMF，剩下的残差 $r = x - \text{IMF}$ 继续下一轮

⑥ 残差单调或极值太少时停止

结果：`x = Σ IMF_i + r`，前面的 IMF 是高频快变，后面的是低频慢变——和傅里叶的"基频+谐波"思路完全不同，它是按时间尺度自适应分层的。

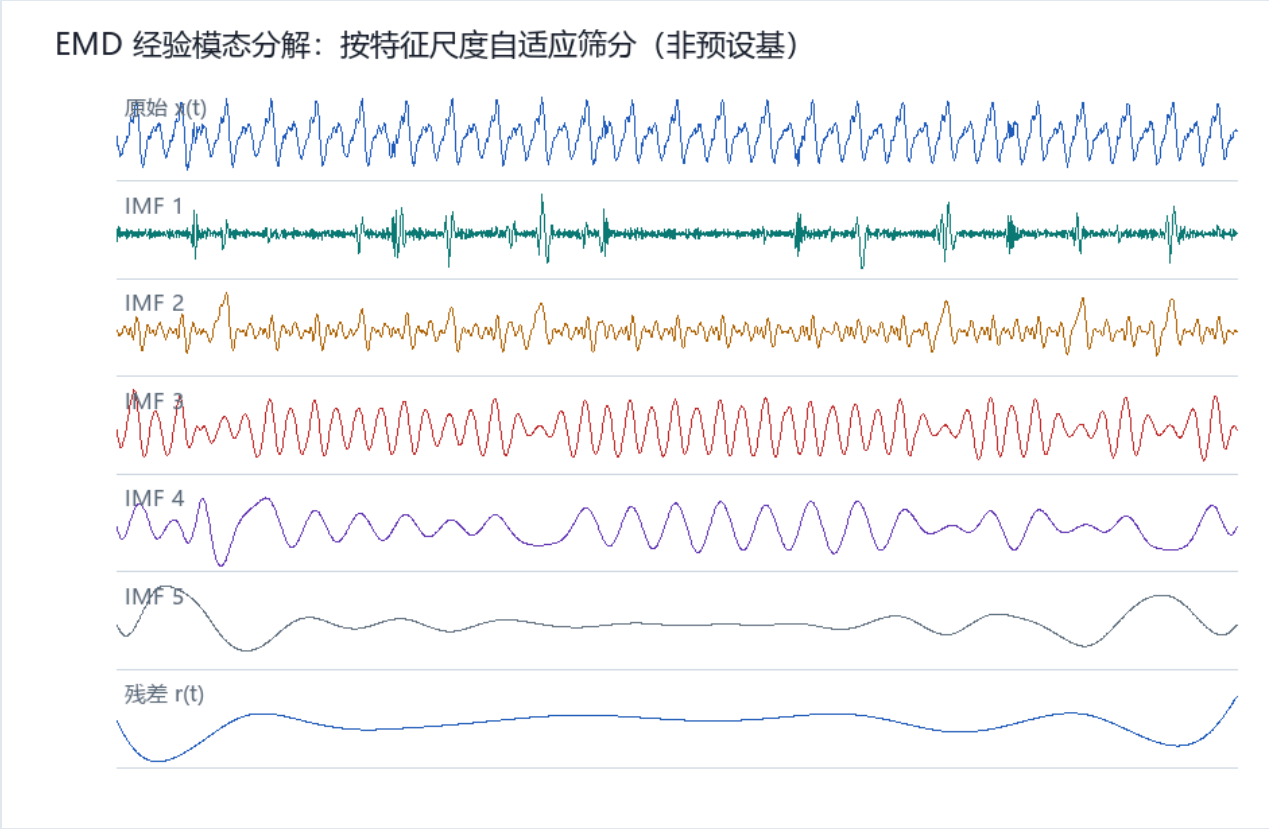


图 7-2　铁芯松动样本的 IMF 分解：从上到下时间尺度由细到粗，最后一行是残差（趋势）。重构所有分量应能还原原信号（误差 1e−17 量级）

EMD 的三个已知问题（必须知道）

表 7-3　EMD 的三个已知问题与缓解办法

问题	表现	缓解办法
端点效应	两端的包络靠外推，误差向内传播，分解越深越严重	端点延拓（镜像、极值延拓）、只取中间段分析
模态混叠	一个 IMF 里混进了差异很大的多个尺度，或同一尺度被拆到两个 IMF	EEMD（加噪声多次平均）、CEEMDAN
停止准则主观	筛分次数没有客观标准，12 次和 20 次结果不同	固定筛分次数或用标准差准则（SD < 0.2~0.3）

另外要清楚：EMD 没有傅里叶那样严格的正交基理论，它的"正交性"只是近似指标（本项目实测正交指数约 0.07），不要拿"正交分解"的结论去套它。

⑤ 三者怎么选

表 7-4　三种时频方法的选择依据

方法	基函数	分辨率分配	计算量	适用
STFT	固定（正弦）	固定	低	谐波结构分析、实时系统
小波	需人工选母小波	高频短窗/低频长窗	中	瞬态检测、突变定位、去噪

方法	基函数	分辨率分配	计算量	适用
EMD	由数据自适应决定	自适应	高（要反复插值）	非平稳、非线性、未知形态的信号

给声纹场景的建议：要定位"脉冲什么时候出现"用短窗 **STFT** 或小波；要分析"谐波比例"用长窗 **STFT** 或直接 **FFT**；要处理形态未知的复杂非平稳信号再考虑 **EMD**。文献里（如李楠 2023 用"小波变换生成声纹特征图谱"再做多传感器融合）走的就是小波路线，因为松动故障的关键是撞击瞬态。

误区与注意事项

误区 ① 以为小波能突破不确定原理。它只是"按频率分配分辨率"，乘积下界不变。② 把 **CWT** 的横轴当线性频率。尺度轴是对数的，直接标线性频率会把 100 Hz 标成 900 Hz（本项目真的踩过这个坑）。③ **EMD** 分解越深越好。越深端点效应越重，IMF 越多不等于信息越多。④ 拿 **EMD** 的正交性当定论。它只是经验算法。

自检

- [ ] 我能说出 STFT 的固定窗在什么场景下会失效。
- [ ] 我能用 `f ≈ Fs/a` 把小波尺度换算成频率。
- [ ] 我知道 3 层 DWT 的系数长度与对应频带。
- [ ] 我能说出 EMD 的 IMF 两个条件与三个已知问题。

第 8 讲

神经网络基础：BP 神经网络

目标

- 能写出一个神经元的计算，并说清激活函数为什么必须是非线性的。
- 能自己推出 `∂L/∂W = aᵀ·δ` 这类反向传播公式，而不是背下来。
- 会做数值梯度检查，能看懂"误差 1e−11"意味着什么。

产出

- 一个纯 numpy 的两层 BPNN，前向 + 反向都能跑通。
- 梯度检查的最大绝对误差小于 1e−8。

配套可拖演示 打开同目录下的 `labs.html`（第二个标签页）：9 个权重都能拖，右侧实时给出这一串链式相乘的每一项数值，并和"左右各挪 1e-4 算出的数值梯度"对照——两者对不上，就说明链式某一步漏了项。

先修知识与准备

需要：矩阵乘法与维度的推理（`(m,n)@(n,p)=(m,p)`）、链式法则、`exp` 与 `log`。若矩阵维度推不清，反向传播一定写不对——这是最常见的能力缺口，建议先做本讲最后的维度练习。

① 从一个神经元说起

$$z = \Sigma\, w_i \cdot x_i + b \qquad \text{（加权和 + 偏置）}$$

$$a = \sigma(z) \qquad \text{（激活）}$$

$$\sigma(z) = 1 \,/\, (1 + e^{-z}) \qquad \text{（sigmoid，输出 (0,1)，可解释为概率）}$$

为什么要有 σ？因为如果只有加权和，多层网络叠加起来仍然是线性的：$W_2(W_1x) = (W_2W_1)x$，两层等于一层，学了也没用。非线性激活是"深度"有意义的前提。这也是为什么 XOR 这种线性不可分问题，单层感知机永远做不对，加一个隐层就能做对。

常用激活函数

表 8-1　常用激活函数及其导数

名称	公式	导数	特点
sigmoid	1/(1+e^−z)	σ(1−σ)	输出概率，两端梯度消失（饱和）
tanh	(e^z−e^−z)/(e^z+e^−z)	1−tanh²	零均值，收敛比 sigmoid 快
ReLU	max(0, z)	z>0 为 1，否则 0	不饱和、计算便宜，深层网络首选

② 前向传播：把一帧特征变成概率

隐层：$h = \sigma(W_1 \cdot x + b_1)$　　x 是 39 维 MFCC 统计量

输出：$z_2 = W_2 \cdot h + b_2$

概率：$p = \mathrm{softmax}(z_2)$，　$p_c = e^{z_{2,c}} / \Sigma_j e^{z_{2,j}}$

softmax 的作用是把任意实数变成"和为 1 的概率"。注意它对所有 z 加同一个常数不变（分子分母同乘），所以实现时要先减最大值防止 `exp` 溢出：`z - z.max()`。

③ 损失函数与那个"漂亮"的梯度

交叉熵损失：$L = -\Sigma_c y_c \cdot \log(p_c)$

y 是 one-hot 标签（正确类为 1，其余 0）。若只有一个正确类 c*，则 $L = -\log(p_{c^*})$

对 softmax + 交叉熵求导，会得到一个极其简洁的结果：

$\partial L/\partial z_2 = p - y$

例：三类，输出 $p = [0.7, 0.2, 0.1]$，真实标签 $y = [0, 1, 0]$

$\partial L/\partial z_2 = [0.7, -0.8, 0.1]$

含义：预测概率比标签大的往下压（正梯度），比标签小的往上抬（负梯度）

这个结果不是巧合，而是"指数族 + 对数损失"的共轭性质。知道它，反向传播的第一层就不用推了。

④ 反向传播：逐层往回传

记 $\delta_2 = \partial L/\partial z_2 = p - y$，则：

$\partial L/\partial W_2 = \delta_2 \cdot h^T$　　（形状：输出维 × 隐层维，与 W_2 一致）

$\partial L/\partial b_2 = \delta_2$

$\delta_1 = \partial L/\partial z_1 = (W_2^T \cdot \delta_2) \odot \sigma'(z_1)$　　$\odot$ 是逐元素乘

$\partial L/\partial W_1 = \delta_1 \cdot x^T$

$\partial L/\partial b_1 = \delta_1$

三句话记住结构：对权重求导 = 该层的 **δ** 乘上一层的输入（外积）；把 **δ** 往回传 = 用 **W** 的转置乘 **δ**，再乘本层激活的导数。

维度是唯一的检查手段　如果 $\partial L/\partial W_1$ 的形状和 W_1 不一样，一定是转置放错了。W_1 是 `(h, d)`，`x` 是 `(d,)`，δ_1 是 `(h,)`，所以必须写 `δ1[:,None] @ x[None,:]` 得到 `(h,d)`。批量时 `X` 是 `(B,d)`，则是 `δ1.T @ X`。

⑤ 手算一次参数更新

用最简单的单神经元（sigmoid 输出、二分类、交叉熵）走一遍，数字都能验算：

> 输入 $x = [1, 2]$，权重 $w = [0.5, -0.5]$，偏置 $b = 0$，标签 $y = 1$，学习率 $\eta = 0.1$
>
> 前向：$z = 0.5\times1 + (-0.5)\times2 + 0 = -0.5$
>
> $p = \sigma(-0.5) = 1/(1+e^{0.5}) = 0.3775$
>
> 损失：$L = -\log(0.3775) = 0.974$
>
> 反向：$\delta = p - y = 0.3775 - 1 = -0.6225$
>
> $\partial L/\partial w = \delta \cdot x = [-0.6225, -1.245]$
>
> $\partial L/\partial b = \delta = -0.6225$
>
> 更新：$w \leftarrow w - \eta \cdot \partial L/\partial w = [0.5 + 0.06225, -0.5 + 0.1245] = [0.5623, -0.3755]$
>
> $b \leftarrow 0 + 0.06225 = 0.0623$
>
> 验证：新的 $z = 0.5623\times1 - 0.3755\times2 + 0.0623 = -0.1264$
>
> $p = \sigma(-0.1264) = 0.4685$ ← 从 0.3775 升到 0.4685，朝着标签 1 靠近了 √

这一步算完，你就掌握了训练的全部本质：预测 → 算损失 → 求梯度 → 沿负梯度方向挪一小步。后面所有网络，无非是把这个过程重复几百万次、并且梯度算得更聪明。

⑥ 梯度检查：怎么确认反向传播没写错

解析梯度可能推错，但数值梯度一定对（它就是导数的定义）：

> 数值梯度：$\partial L/\partial w \approx [L(w + \varepsilon) - L(w - \varepsilon)] / (2\varepsilon)$，　ε 取 1e−5（中心差分，误差 $O(\varepsilon^2)$）

对每个参数抽几个探针，比较解析梯度与数值梯度。判据要同时看两个数：

表 8-2　梯度检查的两个判据

指标	含义	合格线
绝对误差 |g_num − g_ana|	梯度值本身差多少（硬判据）	< 1e−8
相对误差 |差|/(|g_num|+|g_ana|)	近零梯度处会被放大，仅供参考	通常 < 1e−4

本项目真实踩过的坑 BPNN 的梯度检查一开始报相对误差 **5.3e−6**，看起来比 CNN 的 4e−11 差五个数量级，怀疑反向传播写错。逐参数排查后发现：误差最大的那个探针，解析梯度只有 −2.03e−6（几乎是 0），绝对误差只有 1.03e−11——是"除以接近 **0** 的数"把相对误差放大了，不是算错。所以别只看相对误差，一定要把绝对误差一起报出来。

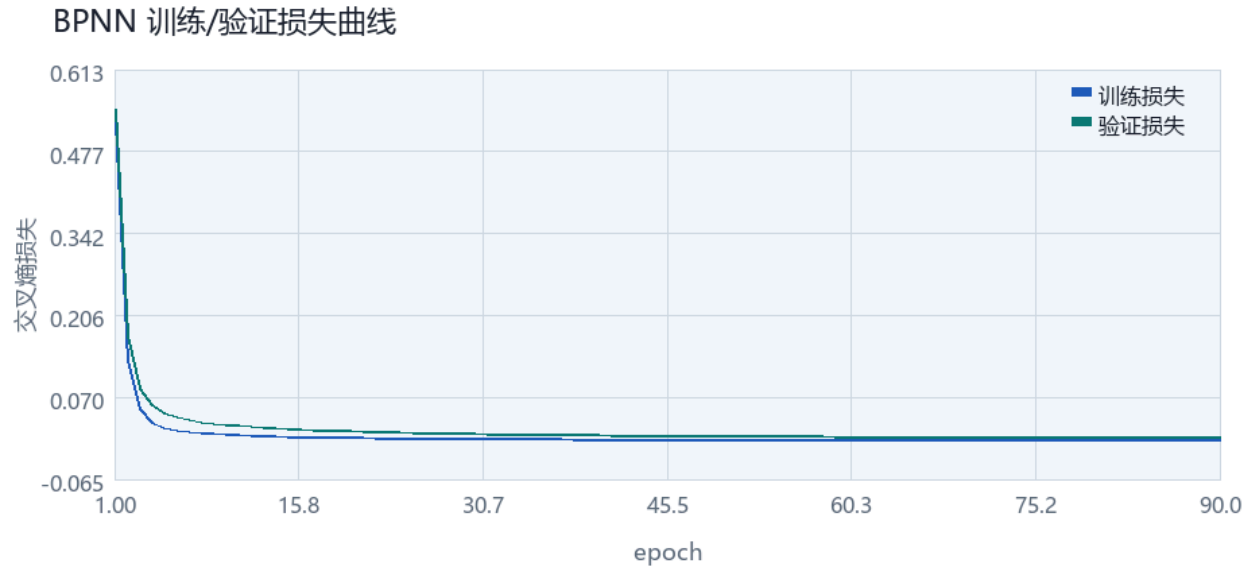


图 8-1　BPNN 的训练/验证损失曲线：前 20 个 epoch 快速下降，之后趋缓。若验证损失开始上升而训练损失继续下降，就是过拟合的信号（第 10 讲）

⑦ 实现要点（纯 numpy）

```
class BPNN:
    def __init__(self, n_in, n_hid, n_out, seed=0):
        r = np.random.default_rng(seed)
        # 初始化不能全零：否则同层所有神经元梯度相同，永远学不出差异
        self.W1 = r.normal(0, np.sqrt(2/n_in),  (n_hid, n_in))
        self.b1 = np.zeros(n_hid)
        self.W2 = r.normal(0, np.sqrt(2/n_hid), (n_out, n_hid))
        self.b2 = np.zeros(n_out)

    def forward(self, X):
        self.z1 = X @ self.W1.T + self.b1
        self.h  = 1/(1+np.exp(-self.z1))
        z2 = self.h @ self.W2.T + self.b2
        z2 -= z2.max(1, keepdims=True)          # 防 exp 溢出
        e = np.exp(z2); self.p = e / e.sum(1, keepdims=True)
        return self.p

    def backward(self, X, Y):
        d2 = (self.p - Y) / len(X)              # δ₂（已含 batch 平均）
        gW2 = d2.T @ self.h
        gb2 = d2.sum(0)
        d1 = (d2 @ self.W2) * self.h * (1 - self.h)   # δ₁
        gW1 = d1.T @ X
        gb1 = d1.sum(0)
        return [gW1, gb1, gW2, gb2]
```

误区与注意事项

误区 ① 权重全零初始化。对称性不破，隐层神经元永远学成一样。② 忘记除 **batch** 大小。梯度随 batch 线性放大，学习率等于白设。③ **softmax** 不减最大值。数值溢出变成 nan。④ 只用相对误差判断梯度。近零梯度会误报（见上文）。⑤ 把 **sigmoid** 用于深层网络。梯度消失，改用 ReLU。

自检

- [ ] 我能写出单个神经元的计算，并解释为什么必须非线性。
- [ ] 我能推出 $\partial L/\partial W_2 = \delta_2 h^\top$ 和 $\delta_1 = (W_2^\top \delta_2) \odot \sigma'(z_1)$。
- [ ] 我能跟着手算完成那次参数更新（w 从 [0.5,−0.5] 变成 [0.5623,−0.3755]）。
- [ ] 我知道梯度检查要看绝对误差，并能解释相对误差为什么会误报。

第 9 讲

卷积神经网络 CNN

目标

- 理解卷积为什么比全连接更适合图像/谱图（三件事：局部、共享、平移）。
- 能算卷积层的输出尺寸和参数量。
- 知道 im2col 是怎么把卷积变成矩阵乘法的。

产出

- 手写一维/二维卷积前向，输出尺寸与手算一致。
- CNN 的数值梯度检查通过（绝对误差 < 1e−8）。

先修知识与准备

需要第 8 讲的反向传播。卷积本身只是"加权求和"，不需要提前学信号处理里的卷积定理。

① 为什么不能直接把谱图拉平喂给全连接

一张 32×32 的 log-Mel 图拉平是 1024 维，接一个 1000 维隐层需要 `1024×1000 ≈ 100 万` 个参数。两个问题：参数太多容易过拟合（尤其声纹样本常常只有几百条）；更重要的是，拉平破坏了空间结构——第 3 行第 5 列与第 4 行第 5 列本来是相邻的时频点，拉平后这个"相邻"关系就丢了。

卷积用三个假设换来了极高的参数效率：

表 9-1　卷积的三个假设与收益

假设	含义	带来什么
局部性	相邻的时频点才相关，远处的先不直接相连	每层只看 3×3 这样的小块
平移不变	同一个模式出现在图的左边或右边，都该被同一种方式识别	同一个卷积核滑过整张图（权重共享）
层次性	简单的边/纹理组合成复杂的结构	堆叠多层，感受野逐层扩大

② 卷积的运算与尺寸

> $y[i, j] = \Sigma_p \Sigma_q \, x[i+p, j+q] \cdot K[p, q] + b$
>
> K 是卷积核（如 3×3），b 是偏置。核在输入上滑动，每停一次算一次加权和

输出尺寸（必须背下来）：

> $H_{out} = \text{floor}((H_{in} - F + 2P) / S) + 1$ （宽同理）
> F：核大小，P：补零圈数，S：步长
> 例：32×32 输入，3×3 核，P=1，S=1 → (32−3+2)/1+1 = 32（尺寸不变，称 same padding）
> 例：32×32 输入，3×3 核，P=0，S=2 → (32−3)/2+1 = 15（下采样）

参数量（也是必考）：

> 参数 = $(F \times F \times C_{in} + 1) \times C_{out}$
> 例：输入 1 通道，4 个 3×3 核 → (3×3×1 + 1)×4 = 40 个参数
> 对比全连接：1024 → 1000 需要 1,025,000 个参数，差 25000 倍

40 个参数就能提取 4 种局部模式——这就是卷积"参数效率"的直观含义。当然 40 个参数能表达的东西有限，所以实际网络会堆很多层、很多通道（ResNet 有上千万参数），但相对全连接仍然是数量级的节省。

③ 池化与感受野

> 最大池化：y = max(x[i:i+2, j:j+2]) （2×2、步长 2 → 尺寸减半）
> 平均池化：y = mean(...)

池化的作用是降维 + 提供一定的小范围平移鲁棒性（模式挪动一两个像素，最大值还在窗内）。代价是丢失精确定位信息——所以检测任务里要慎用大池化。

感受野指"输出上的一个点，能看到输入的多大范围"。3 层 3×3 卷积（步长 1）的感受野是：

> 1 层：3×3
> 2 层：3 + (3−1) = 5×5
> 3 层：5 + (3−1) = 7×7
> 一般式：$RF_k = RF_{k-1} + (F - 1) \cdot \Pi$(前面各层步长)

④ im2col：把卷积写成一次矩阵乘法

直接用四重循环算卷积在 numpy 里极慢。工业界的做法是把每个"待卷积的小块"拉成一列，拼成矩阵，然后一次 `@` 算完：

> 输入 (C, H, W) → 取出所有 F×F 的小块 → 拼成 $(C \cdot F \cdot F, H_{out} \cdot W_{out})$ 的矩阵 X_{col}
> 卷积核展开成 $(C_{out}, C \cdot F \cdot F)$ 的矩阵 W_{col}
> 输出 = $W_{col} @ X_{col}$ → reshape 成 $(C_{out}, H_{out}, W_{out})$

这样就把卷积变成了 GEMM（通用矩阵乘法），可以吃满 BLAS 的性能。反向传播也随之变得简单：

> 前向：$\text{out} = W_{\text{col}} @ X_{\text{col}}$
>
> 反向：$dW_{\text{col}} = \text{dout} @ X_{\text{col}}^{T}$
>
> $dX_{\text{col}} = W_{\text{col}}^{T} @ \text{dout}$ → 再用 col2im 还原成 (C,H,W)

注意 `col2im` 不能简单 reshape，因为相邻的小块是重叠的，重叠位置的梯度要累加而不是覆盖。这是手写 CNN 时第二个常见错误（第一个是池化的梯度分配）。

⑤ 为什么 CNN 特别适合谱图

一张 log-Mel 谱图上，"局部模式"恰好对应真实的物理结构：

表 9-2　谱图局部模式与物理含义的对应

图上看到的局部模式	物理含义	CNN 学到的东西
一小段竖直亮条	瞬态脉冲（局放、撞击）	第 1–2 层的边缘检测器
几条平行的水平线	谐波族	第 2–3 层的"条纹组合"检测器
低频段的一小片结构	50 Hz 倍频族（偏磁）	深层的高层语义组合

这也是文献里最常见的做法：张重远 2020 把 Mel 时频谱直接送进 CNN，铁芯三种工况识别率 99.71%；陆云才 2023 把时域与频域信号做成多通道输入，配合集成学习，单一缺陷 99.2%、混合缺陷 99.7%。

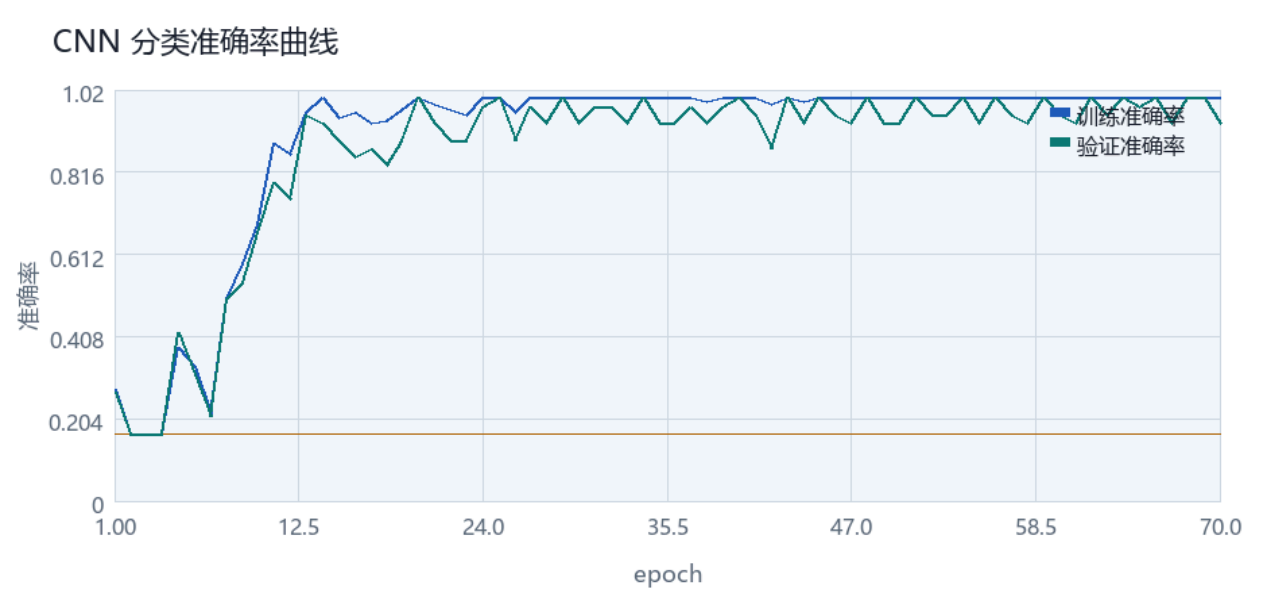


图 9-1　手写 CNN 在 6 类合成声纹上的准确率曲线：训练集很快到 100%，验证集在 97% 附近波动——验证集低于训练集是正常现象，差距过大才是过拟合

⑥ 实现要点

```
def im2col(X, F, stride):
    C, H, W = X.shape
    Ho = (H - F)//stride + 1; Wo = (W - F)//stride + 1
    cols = np.empty((C*F*F, Ho*Wo))
    for c in range(C):
        for i in range(F):
            for j in range(F):
                patch = X[c, i:i+stride*Ho:stride, j:j+stride*Wo:stride]
                cols[(c*F+i)*F+j] = patch.ravel()
    return cols, (C, H, W, Ho, Wo)

# 前向：out = (W_col @ cols).reshape(C_out, Ho, Wo) + b[:,None,None]
```

维度检查：`W_col` 是 `(C_out, C·F·F)`，`cols` 是 `(C·F·F, Ho·Wo)`，乘出来 `(C_out, Ho·Wo)`，reshape 成 `(C_out, Ho, Wo)`——和手算的输出尺寸对上就说明写对了。

误区与注意事项

误区 ① 输出尺寸算错。永远先用公式算一遍再跑代码，别靠"看起来差不多"。② **col2im** 时直接 **reshape**。重叠位置必须累加。③ 池化反向传播把梯度全给最大值位置还是平均分？最大池化只把梯度给最大值那一个位置，其余为 0。④ 以为卷积核越大越好。两个 3×3 的感受野等于一个 5×5，但参数更少、非线性更多，所以现代网络偏爱小核堆叠。

自检

- [ ] 我能说出卷积相对全连接的三个假设，以及它带来的参数节省。
- [ ] 我能算出 32×32、3×3 核、P=1、S=1 的输出尺寸（32）和参数量（4 个核 → 40）。
- [ ] 我知道 3 层 3×3 卷积的感受野是 7×7，并能推出一般式。
- [ ] 我能解释 im2col 的作用，以及 col2im 为什么要累加。

第 10 讲

训练、评估与过拟合

目标

- 会正确划分数据、正确做标准化，能识别"数据泄漏"。
- 能读懂混淆矩阵、Precision/Recall/F1、ROC/AUC。
- 理解为什么"100% 准确率"常常是坏消息。

产出

- 一份完整的评估报告：混淆矩阵 + 逐类 F1 + 一条噪声衰减曲线。
- 能对一个"99% 准确率"的结论提出至少两个质疑。

① 数据怎么分，为什么必须这样分

> 训练集 60%：用来更新参数
>
> 验证集 20%：用来选超参数（学习率、网络层数、早停轮数）
>
> 测试集 20%：只在最后用一次，报告性能

三层划分的意义：如果用测试集去调超参数，测试集的信息就"泄漏"进模型了，最后报出来的数字是虚高的。测试集只能看一次，看多了它就不再是"没见过的数据"。

数据泄漏的典型例子（都真实发生过）

表 10-1　数据泄漏的典型例子

做法	为什么错	正确做法
先对全体数据标准化再划分	测试集的均值方差泄漏进训练	只用训练集算 mean/std，再变换验证/测试集
同一段录音切成多段，分别放进训练和测试	相邻片段几乎一样，等于考原题	按"录音/设备/日期"分组划分
用测试集反复调参	测试集变成了验证集	调参只看验证集

第二行在声纹里极其常见：把一段 10 秒录音切成 20 个 0.5 秒样本，随机划分后训练集和测试集里都有来自同一段录音的片段——模型只要记住"这段录音的底噪"，就能答对，看起来准确率 99%，换个设备立刻崩。

② 过拟合：训练集满分，测试集不及格

过拟合指模型把训练集的噪声也学进去了。判断信号很直接：

> 训练损失持续下降，验证损失先降后升 → 过拟合
>
> 训练与验证损失都很高 → 欠拟合（模型太小/训练不够）

四类应对手段

表 10-2 过拟合的四类应对手段

手段	怎么做	代价
L2 正则	损失加 $\lambda\Sigma w^2$，抑制大权重	需要调 λ
Dropout	训练时随机丢弃一部分神经元	训练变慢、推理要关闭
早停	验证损失不降就停	需要可靠的验证集
数据增强	加噪声、时间平移、增益扰动	可能引入不真实样本

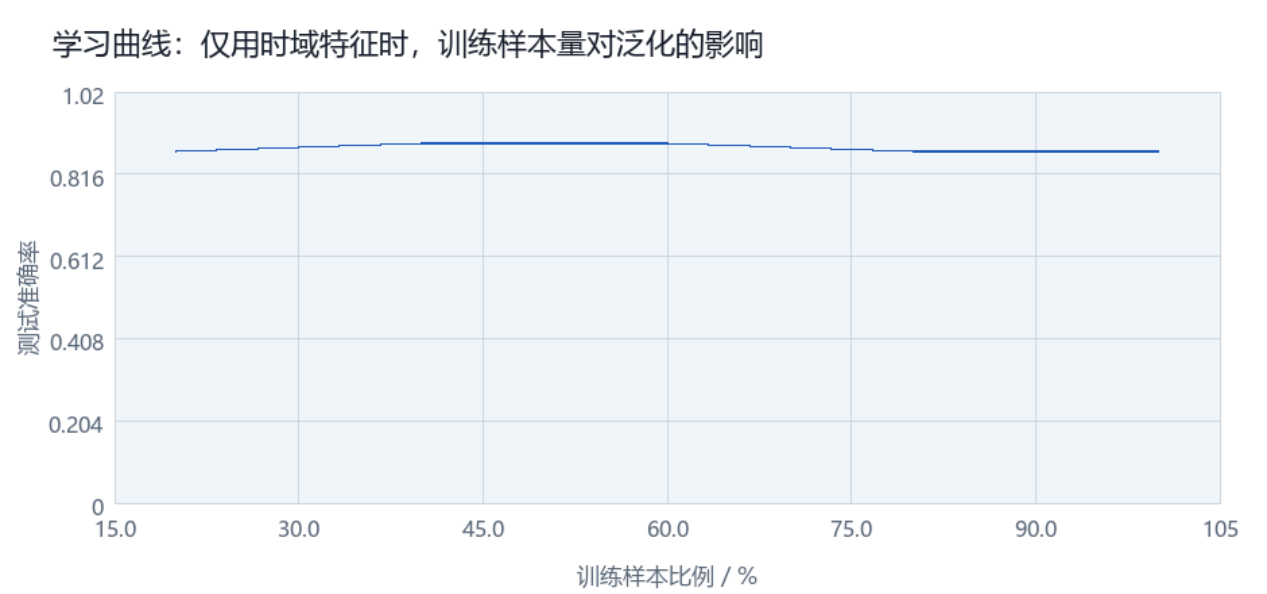


图 10-1 学习曲线：只给模型 8 维时域特征，随训练样本比例增加，测试准确率的变化。样本少时曲线抖动大——这就是声纹领域为什么总在喊"数据不够"

③ 准确率会骗人：混淆矩阵与 F1

假设 100 个样本里 95 个是"正常"、5 个是"局放"。一个把所有样本都判成"正常"的模型，准确率是 **95%**，但它一个故障都抓不到。所以必须看混淆矩阵：

表 10-3 二分类混淆矩阵

	预测：正常	预测：故障
真实：正常	TN（真阴）	FP（假阳，误报）
真实：故障	FN（假阴，漏报）	TP（真阳，正确报警）

> 准确率 Accuracy = (TP+TN) / 全部
>
> 精确率 Precision = TP / (TP+FP)　报了警的里面有多少是真故障（少误报）
>
> 召回率 Recall　= TP / (TP+FN)　真故障里抓到了多少（少漏报）
>
> F1 = 2·P·R / (P+R)　　两者的调和平均

工程上怎么选？漏报代价高（安全类）就提高召回率，误报代价高（人力巡检）就提高精确率。这也解释了第 12 讲的 DCF：它把两类错误的代价直接写进指标。

④ ROC 与 AUC：把阈值的影响一次画完

分类器输出的其实是概率，取不同的阈值就得到不同的 (FPR, TPR) 点。把所有阈值扫一遍连成曲线，就是 ROC：

> FPR = FP/(FP+TN)（假阳率）　TPR = TP/(TP+FN)（真阳率 = 召回率）
>
> AUC：曲线下面积。0.5 = 随机猜，1.0 = 完美
>
> 注意：这是机器学习教材的画法。声纹领域的传统画法是横轴 FA、纵轴 FR 的递减曲线，AUC 越小越好——第 12 讲会专门讲这个陷阱的画法。声纹领域的传统画法是横轴 FA、纵轴 FR 的递减曲线，AUC 越小越好——第 12 讲会专门讲这个陷阱

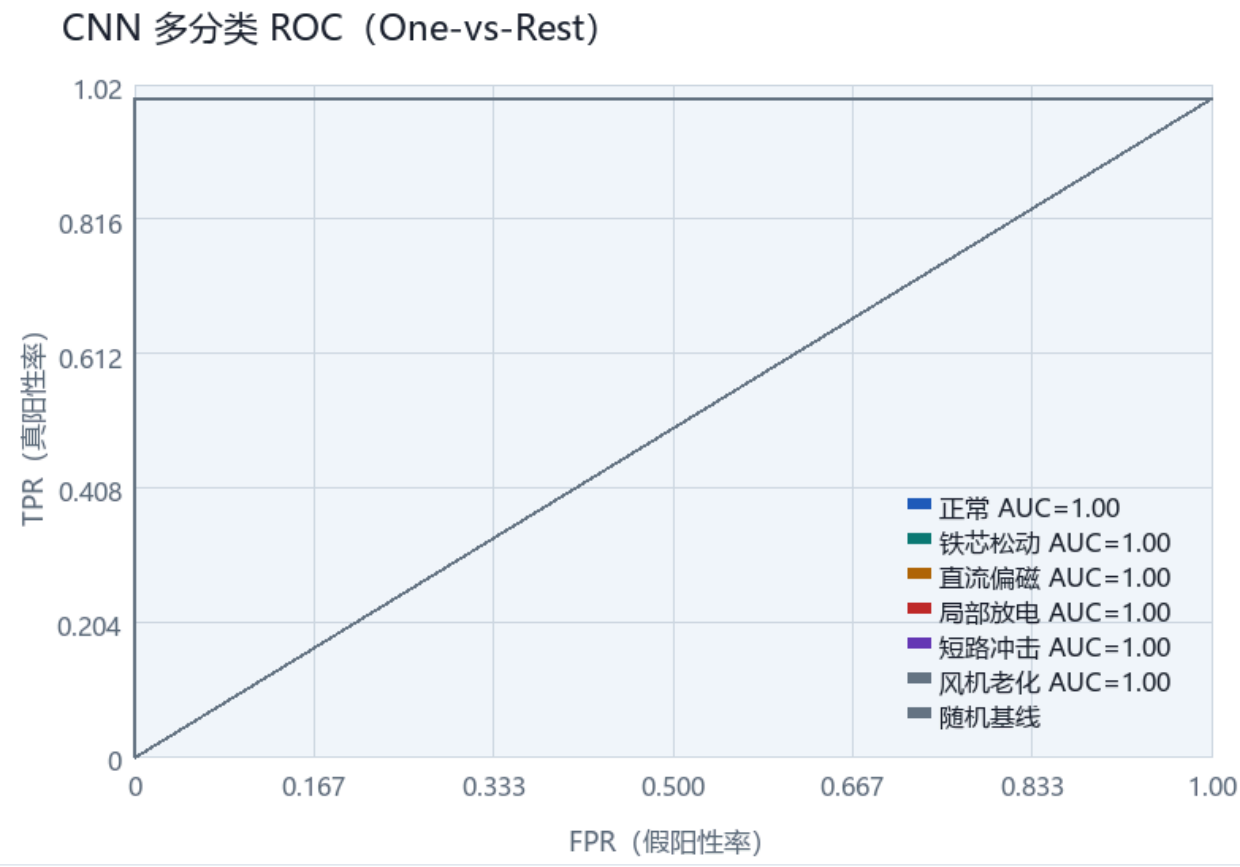


图 10-2　六类声纹的 One-vs-Rest ROC：全部 AUC = 1.00，曲线贴住左上角。这不是好消息，而是数据太容易——合成数据可分性过高，ROC 失去区分度

从这张图该学到的事 当所有模型的 AUC 都等于 1 时，ROC 就没有信息量了。这时要么换更难的数据（加噪声、跨设备），要么换指标（噪声鲁棒性曲线、混淆矩阵里的具体混淆对）。本项目正是因此补了一张"噪声衰减曲线"。

⑤ 比准确率更值得报告的：鲁棒性曲线

做法：给测试样本逐级注入高斯噪声（噪声标准差 / 数据标准差 = 0, 0.34, 0.68 … 2.0），看准确率怎么掉。

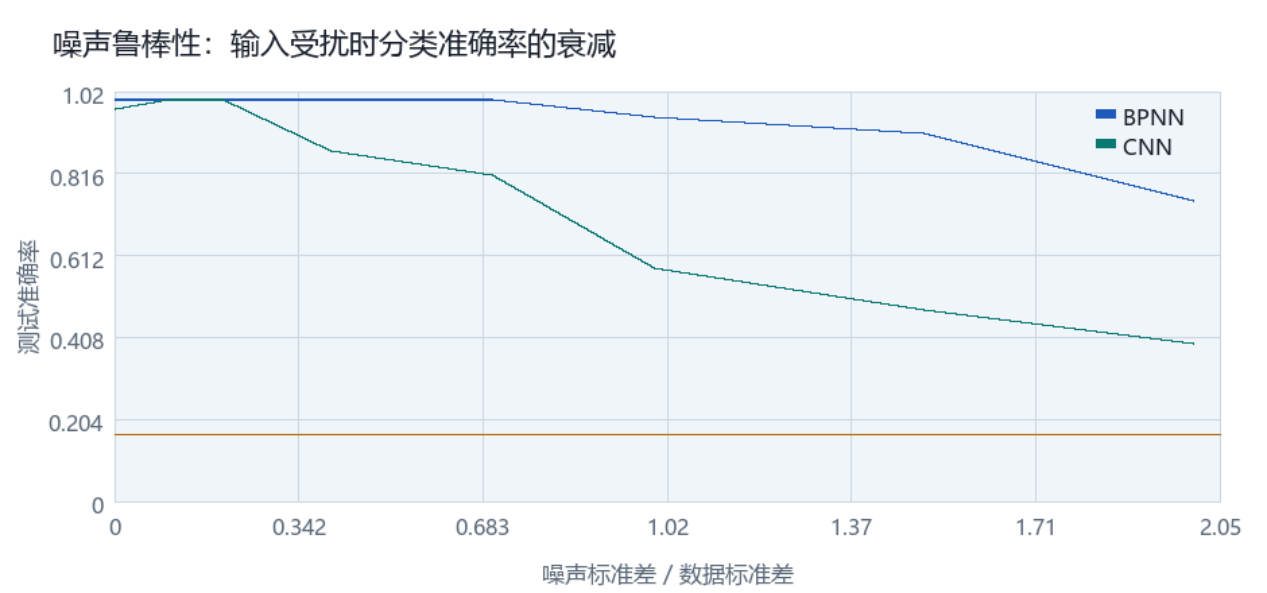


图 10-3　噪声鲁棒性曲线：噪声比 1.0 时 BPNN 仍有 96%，CNN 掉到 58%；噪声比 2.0 时分别是 75% 与 40%。**CNN** 更依赖干净的谱图纹理，**BPNN** 依赖更粗的统计量，所以抗噪表现不同

为什么这条曲线比"100% 准确率"有价值？因为现场永远有噪声。一个在干净数据上 99%、加噪后 50% 的模型，实际不可用；而在干净数据上 95%、加噪后 85% 的模型才是能落地的。文献里也是这么做的：李嘉宁 2025 明确报告"高信噪比 96%、10 dB 时提升 9.4%"，而不是只报最好的那个数。

误区与注意事项

误区 ① 只报准确率。类别不平衡时毫无意义。② 先标准化再划分。数据泄漏。③ 同一录音切段后随机划分。等于考原题。④ 用测试集调参。⑤ 看到 **AUC=1** 就高兴。先怀疑数据太简单。

自检

- [ ] 我能说出训练/验证/测试三者的分工，并举一个数据泄漏的例子。
- [ ] 我能写出 Precision、Recall、F1 的公式，并说出什么时候该优先哪个。
- [ ] 我能解释为什么"全部判正常"也能有 95% 准确率。
- [ ] 我能说清"噪声鲁棒性曲线"比单一准确率更有价值的原因。

第 11 讲

声纹识别概览：任务、流程、与语音识别的区别

目标

- 能区分声纹验证、辨认、开集/闭集四种设定。
- 能说清"声纹识别"和"语音识别"为什么是正交的两个问题。
- 能把人的声纹任务与设备状态识别一一对应起来。

产出

- 一张"任务类型 → 判定方式 → 指标"的对照表。
- 能对任意一个声纹需求说出该用哪种设定。

① 四种任务设定，先分清再谈算法

表 11-1　四种任务设定的对比

设定	问题	本质	典型指标
声纹验证（1:1）	这段话是"张三"说的吗？	二值判别（是/否）	EER、DCF、FAR/FRR
闭集辨认（1:N）	这段话是库里 N 个人中的谁？	N 分类（一定有答案）	准确率、Top-1/Top-5
开集辨认	这段话是库里的人吗？是谁？	先拒识再分类	EER + 准确率
声纹分割/聚类	这段录音里有几个人，各自说了哪段？	无监督聚类	DER（分割错误率）

为什么必须先分清？因为指标完全不同、算法取舍也完全不同。验证任务关心"阈值怎么定"，辨认任务关心"排名对不对"，开集还多一个"拒识"环节。拿辨认的准确率去评价一个验证系统，是没有意义的。

② 与语音识别正交：一句话讲透

> 语音识别：想知道"说了什么"，要滤掉说话人信息，只保留文本内容
>
> 声纹识别：想知道"谁说的"，要说话人信息，只保留文本内容
>
> 声纹识别：想知道"谁说的"，要滤掉文本内容，只保留说话人身份
>
> 方向完全相反，所以叫"正交"文本内容，只保留说话人身份
>
> 方向完全相反，所以叫"正交"

这个说法（王泉《声纹技术》3.1.1 的表述）非常好用，因为它直接解释了工程上的差异：语音识别要做"说话人自适应"来消除个体差异；声纹识别反而要做"内容归一化"来消除文本差异。同一个 MFCC 特征，两个任务都在用，但后面的建模目标相反。

③ 系统流程：注册 + 测试

> 注册（enrollment）：用户说几段话 → 提特征 → 训练/存储该用户的模型（或一个 embedding 向量）
>
> 测试（test）：说一段话 → 提特征 → 与库里的模型逐一比较 → 得到相似度分数
>
> 判决：分数 > 阈值 → 接受；否则拒绝

要注意：分数不是概率，它只是一个"越像越大"的数。阈值定在哪里，取决于你把"误拒"和"误受"哪个看得更重——这正是第 12 讲的全部内容。

文本相关 vs 文本无关

表 11-2　文本相关与文本无关声纹的对比

类型	做法	优点	缺点
文本相关	固定说同一句话（如"打开门禁"）	性能好，可做防录音攻击	用户体验差，易被录音重放
文本无关	随便说什么	自然，防重放	难度大，需建模内容无关的身份特征
文本提示	系统随机给词，用户跟读	兼顾性能与防攻击	实现复杂

④ 声纹到底"看"的是什么生理/行为特征

表 11-3　声纹依赖的生理与行为特征

层面	具体特征	在频谱上的表现
声道长度与形状	声道短 → 共振峰整体偏高	频谱包络的整体平移
声带特性	基频（男 100–150 Hz，女 180–250 Hz）	谐波间隔
发音习惯	语速、连读、口音	时间维度上的动态模式
鼻腔/口腔耦合	个人音色	特定频段的共振峰形状

所以 MFCC 之所以有效，不是因为它"神奇"，而是因为它的低频密采样 + **DCT** 包络恰好抓住了"声道形状"这个身份载体。

⑤ 迁移到设备：人 ↔ 变压器

这套框架能几乎原样搬到设备状态识别上，只要做一次"语义替换"：

表 11-4　人声纹与设备声纹的对应关系

人的声纹	变压器声纹
说话人身份（谁）	运行工况/故障类型（哪种状态）
注册（录入用户声音）	建立各类工况的样本库
文本内容（要消除的干扰）	负载电流、电压波动（要消除的干扰）

人的声纹	变压器声纹
信道差异（麦克风、电话线）	传感器型号、安装位置、传声路径
EER（等错误率）	同样适用，用来定报警阈值

文献里的工作正是这么做的：刘云鹏 2020 用"50 Hz 倍频倒谱系数 + GRU"识别变压器直流偏磁（特征直接利用"偏磁会产生 50 Hz 倍频"这一物理先验）；齐子豪 2025 用多粒度知识特征 + Transformer 做变压器故障声纹辨识。它们都在做同一件事：把人声纹的成熟框架，配上设备特有的物理先验。

误区与注意事项

误区 ① 把辨认的准确率当验证的指标。② 以为分数可以当概率用。未做归一化的分数没有概率意义（第 12 讲的 z-norm/t-norm 就是为此）。③ 忽略信道差异。换麦克风、换距离，性能可能掉一半。④ 用"同一段录音切段"做测试。这是声纹评估里最致命的错误。

自检

- [ ] 我能分清验证、闭集辨认、开集辨认，并说出各自指标。
- [ ] 我能用"正交"解释声纹识别与语音识别的区别。
- [ ] 我能把"注册—测试—阈值判决"流程复述一遍。
- [ ] 我能说出人声纹里"文本内容"对应设备声纹里的什么干扰。

第 12 讲

声纹评价指标：EER、ROC、DCF

目标

- 能画出阈值移动时 FA 与 FR 的反向变化，并解释为什么不存在"两全"的阈值。
- 理解 EER 的定义，并能解释为什么阈值点的真实错误率是 2×EER。
- 会用 DCF 把"两类错误代价不同"写进指标。

产出

- 一张 FA–FR 曲线，并标出 EER 点。
- 给定代价参数能算出 DCF，并说明阈值该往哪边调。

① 两类错误，此消彼长

> FA（False Accept，错误接受 / 误受）：把冒充者当成本人 → 安全风险
>
> FR（False Reject，错误拒绝 / 误拒）：把本人拦在门外 → 体验问题
>
> FAR = FA 次数 / 冒充试验总次数
>
> FRR = FR 次数 / 真实试验总次数

关键事实：它们是同一个阈值的两种后果。把阈值调高（更严格）：更多真实用户被拒（FR 上升），冒充者更难通过（FA 下降）。把阈值调低则相反。所以"既要又不"是不可能的——只能根据代价选一个工作点。

> **一个具体场景** 门禁：误拒的代价是员工迟到抱怨，误受的代价是陌生人进楼。所以门禁阈值通常调得较严（宁可误拒）。手机解锁：误拒的代价是机主烦躁，误受的代价是被人翻手机——两者都重要，所以厂商通常给出"更快/更安全"两档。

② EER：把整条曲线压成一个数

既然阈值可选，就需要一个"与阈值无关"的指标来比较系统好坏。EER（等错误率）就是那条曲线上 **FA = FR** 的点：

> EER = 使 $FA(\theta) = FR(\theta)$ 成立的那个错误率
>
> EER 越小，系统整体越好。EER = 1% 意味着在该点上两个方向的错误率都是 1%

③ 为什么真实错误率是 2×EER（这一节几乎人人误解）

EER 是"每个方向各 1%"。但一次实际使用中，用户要么是本人、要么是冒充者，两者不会同时发生——所以在 **EER** 工作点上，把所有测试合并统计，出错的比例是 **FA + FR = 2×EER**。

> EER = 5% → 在 EER 阈值下，约 10% 的测试会被判错
>
> （其中约一半是本人被拒，一半是冒充被放）
>
> 这不是文字游戏：厂商宣传"EER 5%"，用户实际感受到的出错率是 10%

更严格地说，若真实试验与冒充试验数量相等，总体错误率 = (FA + FR)/2 = EER；若按"每次使用只有一次判决"统计并且两类试验各占一半，则两类错误合计占比为 2×EER。工程上报告指标时务必写清口径：是"每类各 **5%**"，还是"总体 **10%**"。王书 3.2.4 特意点出这一点，就是为了防止误读。

④ ROC 的两种画法（看论文时必须先确认）

表 12-1　ROC 的两种画法对比

画法	横轴	纵轴	曲线走向	AUC 含义
机器学习教材	FPR（=FA）	TPR（=1−FR）	单调递增，贴左上角	越接近 1 越好
声纹/生物识别传统	FA	FR	单调递减，贴左下角	越接近 0 越好

两者是同一组数据的两种视角（纵轴一个用 TPR 一个用 FR = 1−TPR），但结论的读法完全相反。看到一张 ROC 先看轴标再下判断，这是读声纹论文的第一道防线。

⑤ DCF：把"代价"写进指标

EER 假设两类错误代价相等，但现实不是。检测代价函数（Detection Cost Function）把代价与先验概率都显式写进去：

> $C_{det} = C_{FR} \cdot FR \cdot P_target + C_{FA} \cdot FA \cdot P_nontarget$
>
> C_{FR}：漏掉一个真人的代价；C_{FA}：放进一个冒充者的代价
>
> P_target：真实用户出现的先验；P_nontarget：冒充者出现的先验
>
> 选让 C_{det} 最小的阈值 → 这个最小值就是 minDCF

算一个例子：设 C_FR = 1、C_FA = 10（放进冒充者的代价是误拒的 10 倍）、P_target = 0.9、P_nontarget = 0.1。

> 某阈值下 FA = 0.05、FR = 0.02：
>
> C_{det} = 1×0.02×0.9 + 10×0.05×0.1 = 0.018 + 0.050 = 0.068
>
> 若换个更严格的阈值：FA = 0.01、FR = 0.08：
>
> C_{det} = 1×0.08×0.9 + 10×0.01×0.1 = 0.072 + 0.010 = 0.082 ← 反而更差
>
> 说明：在这个代价设置下，第一个阈值更优

看这个例子就明白为什么 EER 不够用：两个阈值的 EER 附近可能差不多，但按实际代价算出来的最优工作点完全不同。

⑥ 分数归一化：z-norm 与 t-norm

同一套系统，换一个说话人/换一段录音，分数分布会整体平移。若用一个固定阈值，就会出现"对某人特别宽松、对某人特别严格"。解决办法是用同一人的其他数据把分数标准化：

> z-norm：用一批"冒充者"对该模型的打分估计均值 μ、标准差 σ，归一化 $s' = (s - \mu)/\sigma$
>
> t-norm：用该条测试语音对一批"背景模型"的打分做归一化
>
> zt-norm：两者串起来，效果通常最好，代价是计算量翻倍

⑦ 三个指标的分工

表 12-2　三个评价指标的分工

指标	回答什么问题	什么时候用	局限
EER	系统整体可分性如何	横向比较算法	忽略代价与先验，且需注明是"每类"还是"总体"
minDCF	在给定代价下最好能做到多好	工程选型、定阈值	依赖代价参数的主观设定
固定阈值下的 FAR/FRR	我实际部署的这个点表现如何	上线验收	不能反映其他工作点

误区与注意事项

误区 ① 把 **EER** 当"总体错误率"。它是单方向错误率，总体约为 2 倍。② 用机器学习教材的 **ROC** 读法去读声纹 **ROC**。轴和结论都相反。③ 拿 **EER** 直接定阈值。EER 点只是"两类等错"，不一定是你的最优工作点，要用 DCF。④ 不做分数归一化就比较不同人的性能。

自检

- [ ] 我能解释为什么提高阈值会同时降低 FA、提高 FR。
- [ ] 我能说清 EER=5% 时，总体错误率为什么约 10%。
- [ ] 我能区分两种 ROC 画法，并说出各自的 AUC 读法。
- [ ] 我能用 `C_det = C_FR·FR·P_T + C_FA·FA·P_I` 算出一个具体数值并比较两个阈值。

第 13 讲

经典声纹方法：GMM、UBM、i-vector

目标

- 理解 GMM 为什么能刻画"一帧特征的概率分布"，以及 EM 在优化什么。
- 理解 UBM-MAP 为什么能解决"每人只有几十秒语音"的问题。
- 知道 i-vector 与 PLDA 各解决了什么，为什么深度学习方法取代了它们。

产出

- 能画出"特征 → GMM/UBM → 超矢量/i-vector → 打分"的流程。
- 能说出每一步的维度数字（39 → 512 高斯 → 400 维 i-vector）。

① 为什么用概率模型而不是直接比特征

一个人说 10 秒话，得到约 1000 帧、每帧 39 维。你没法"直接比较"两个人的 1000×39 矩阵——长度不同、时间对不齐。所以需要一个把变长序列压成定长模型的办法，这就是 GMM 的作用：它用一组高斯分布描述"这个人的特征在空间中倾向出现在哪些区域"。

> $p(x \mid \lambda) = \Sigma_{c=1}^{C} \pi_c \cdot N(x \mid \mu_c, \Sigma_c)$
>
> $\lambda = \{\pi_c, \mu_c, \Sigma_c\}$：权重、均值、协方差
>
> $N(x|\mu,\Sigma) = 1/\sqrt{((2\pi)^d|\Sigma|)} \cdot \exp[-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)]$

直觉：单个高斯只能描述"一个团"，而人的特征分布是多峰的（不同音素、不同发声状态），所以用 C 个高斯加权叠加。C 通常取 512~2048。

② EM 算法：怎么把参数学出来

目标：最大化所有帧的对数似然 `Σ_t log p(x_t|λ)`。直接求导没有解析解，所以用迭代的 EM：

> E 步（算责任度）：$\gamma_tc = \pi_c \cdot N(x_t|\mu_c,\Sigma_c) / \Sigma_j \pi_j \cdot N(x_t|\mu_j,\Sigma_j)$
>
> 含义：第 t 帧"属于第 c 个高斯"的概率
>
> M 步（更新参数）：
>
> $\mu_c \leftarrow \Sigma_t \gamma_tc \cdot x_t / \Sigma_t \gamma_tc$
>
> $\Sigma_c \leftarrow \Sigma_t \gamma_tc \cdot (x_t-\mu_c)(x_t-\mu_c)^T / \Sigma_t \gamma_tc$
>
> $\pi_c \leftarrow \Sigma_t \gamma_tc / T$
>
> 反复迭代，直到对数似然不再明显上升

EM 保证每一步都不会让似然下降（这是它的理论性质），但只保证收敛到局部最优，所以初始化很重要（常用 k-means 初始化）。

③ UBM-MAP：解决"每人只有几十秒"

如果给每个说话人单独训一个 GMM，几十秒语音根本不够（参数有几十万个）。解决办法是两步走：

> 第 1 步：用大量不同说话人的数据训一个"通用背景模型"（UBM），它描述"人类语音的通用分布"
>
> 第 2 步：对每个目标说话人，用他自己的数据把 UBM 微调一下（MAP 自适应）：
>
> $\hat{\mu}_c = \alpha_c \cdot \mu_c^{data} + (1-\alpha_c) \cdot \mu_c^{UBM}$
>
> $\alpha_c = n_c / (n_c + r)$，$n_c$ 是该高斯分到的帧数，r 是相关性因子（约 16）一下（MAP 自适应）：
>
> $\hat{\mu}_c = \alpha_c \cdot \mu_c^{data} + (1-\alpha_c) \cdot \mu_c^{UBM}$
>
> $\alpha_c = n_c / (n_c + r)$，$n_c$ 是该高斯分到的帧数，r 是相关性因子（约 16）

`α` 的含意很直观：该高斯分到的数据多（n_c 大），就更相信自己的数据；数据少就更多依赖 UBM。这样既不用从零学几十万参数，又能体现个体差异。

> **韩书 7.2.2 的实测结论** MAP 自适应时，只更新均值（不更新方差和权重）性能最好。原因：均值携带的身份信息最多，而方差在小样本下估计不稳，硬更新反而引入噪声。这条结论至今仍在很多工程实现里沿用。

④ 从超矢量到 i-vector：一次降维

把每个高斯（C = 512）的均值拼起来，得到一个 `512 × 39 = 19968` 维的"超矢量"。维度太高、且大部分变化是信道造成的。i-vector 用一个低秩假设把它压下来：

> $M = m + T \cdot w$
>
> M：说话人的超矢量（19968 维）
>
> m：UBM 的均值超矢量（所有人共用）
>
> T：总变化空间矩阵（19968 × 400），从大量数据学出来
>
> w：i-vector（400 维）—— 这就是新的说话人表示

直观理解：假设"说话人差异 + 信道差异"这些变化虽然维度高，但其实只在一个 400 维的子空间里发生。于是不管什么原因导致的差异，都被压进同一个 400 维向量 `w` 里——所以叫"总变化空间"（total variability）。

⑤ PLDA：把 i-vector 里的"身份"和"信道"分开

i-vector 里混着身份和信道。PLDA（概率线性判别分析）用生成模型把它们拆开：

> $w = \mu + V \cdot y + U \cdot x + \varepsilon$
>
> y：身份因子（同一个人所有录音共享）
>
> x：信道因子（同一段录音内共享）
>
> V、U：对应的子空间；ε：残差

于是"两段语音是否同一人"变成一个似然比检验：共享同一个 **y** 的假设 **vs** 不共享的假设。这个框架在小样本下比直接算余弦距离稳定得多。

⑥ 方法演进与代表性能

表 13-1　声纹方法的演进与代表性能

阶段	方法	核心思想	典型 EER（历史文献）
1990s–2000s	GMM-UBM	UBM + MAP 自适应，似然比打分	约 5–10%
2000s 中	SVM + GMM 超矢量	把超矢量当特征，用判别式分类	约 3–5%
2010s	i-vector + PLDA	低秩总变化空间 + 概率判别	约 1–3%
2014 以后	x-vector（DNN）	端到端学 embedding	约 0.5–2%

要注意这张表的数字只能看趋势，不能横向比较：不同年份、不同数据集（NIST SRE 各年份条件差异很大）、不同时长条件下的结果不可直接比。这个道理和第 15 讲要讲的"综述里的 99% 不能横比"是同一件事。

在设备声纹里，这条演进路线被大幅简化了——因为类别少（几种工况）、样本少，往往直接用 MFCC 统计量 + SVM/随机森林就够了。文献中也有走"物理先验特征 + 时序模型"路线的：刘云鹏 2020 用 50 Hz 倍频倒谱系数配合 GRU 识别直流偏磁，就是把"偏磁产生倍频"这一物理知识直接做进特征，比硬套通用框架更有效。

误区与注意事项

> **误区** ① 以为 **GMM** 的 **C** 越大越好。数据不够时高斯越多越容易过拟合，UBM 常用 512–2048 是数据量决定的。② **i-vector** 维度越高越好。400 维是经验值，太高会引入信道噪声。③ 跳过 **UBM** 直接训每人一个 **GMM**。小样本必崩。④ 拿不同年份的 **EER** 直接比。数据集条件不同。

自检

- [ ] 我能写出 GMM 的混合密度公式，并解释为什么需要多个高斯。
- [ ] 我能说出 EM 的 E 步和 M 步分别在算什么。
- [ ] 我能解释 UBM-MAP 的 α 系数含义，以及为什么只更新均值最好。
- [ ] 我能写出 `M = m + T·w` 并说出 19968 → 400 维的意义。

第 14 讲

深度学习声纹：x-vector 与损失函数

目标

- 理解 x-vector 的核心创新是"统计池化"，它解决了变长序列问题。
- 知道 softmax、AAM-softmax、triplet 三种损失分别在追求什么。
- 能读懂近年变压器声纹文献的做法，并判断其结论的适用范围。

产出

- 能画出 TDNN + 统计池化 + embedding 的结构图。
- 能从一篇声纹论文里抽出"特征 / 模型 / 数据 / 指标"四要素。

① 从 i-vector 到 x-vector：换个方式得到定长向量

i-vector 的思路是"用 GMM 超矢量 + 低秩投影"，需要先训 UBM、再算 Baum-Welch 统计量，流程长且对不上齐。x-vector 直接把这件事交给神经网络：用 TDNN 逐帧提特征，再用统计池化把变长序列压成定长向量。

> 输入：一段语音的 MFCC（T 帧 × 39 维，T 不固定）
>
> TDNN 前几层：逐帧变换，同时利用上下文（类似一维卷积，但跨层连接）
>
> 统计池化：把 T 帧的均值与标准差拼起来
>
> $\mu = (1/T)\Sigma_t h_t \qquad \sigma = \mathrm{sqrt}[(1/T)\Sigma_t (h_t - \mu)^2]$
>
> 输出 = $[\mu; \sigma]$ → 2×隐层维，与 T 无关
>
> 之后接 2 层全连接 → 512 维 embedding → softmax 分类

为什么是"均值+标准差"而不是只取最后一帧？因为语音是变长的，只取最后一帧会丢掉其他时刻的信息；而均值和标准差是对时间顺序不敏感、但对内容敏感的统计量，正好把"这个人说话的总体倾向"压进去。

② 三种损失函数

表 14-1　三种损失函数的对比

损失	公式要点	在追求什么	局限
softmax	L = −log(e^(z_y) / Σ_j e^(z_j))	把类别分开	只要求"可分"，不要求类内紧凑
AAM-softmax	cos(θ+m) 代替 cos(θ)，再乘尺度 s	类内更紧、类间更开（加角度 margin）	margin 需调，训练更慢

损失	公式要点	在追求什么	局限
triplet	max(0, d(a,p) − d(a,n) + m)	直接优化"同类近、异类远"	三元组采样策略影响很大

AAM-softmax 的关键在于：普通 softmax 只要"正确类得分最高"就满意了，哪怕只高一点点。加 **margin** 后，正确类必须比第二名高出 **cos** 意义上的 **m** 弧度才算过关，于是模型被迫把同一类的样本压得更紧。这对"开集"场景（测试时出现训练时没见过的人）特别重要——因为此时你要比的不是分类对不对，而是 embedding 之间的距离。

③ 两种使用范式

> 范式 A（分类）：训练 softmax 分类器，测试时取最大概率的类
>
> 适合：类别固定（几种已知工况）
>
> 范式 B（度量）：训练 embedding，测试时算余弦相似度 / PLDA 打分，再比阈值
>
> 适合：类别会变（新用户注册、未知故障）

设备声纹大多用范式 A（工况种类固定），人的声纹大多用范式 B（要支持随时新增用户）。这个区别决定了你该用哪种损失：范式 **A** 用 **softmax** 就够，范式 **B** 才需要 **AAM-softmax / triplet**。

手写 BPNN vs 手写 CNN（同一批 6 类声纹）

模型	测试准确率	宏平均 F1	宏平均 AUC	参数量	训练耗时	最终验证准确率
BPNN	1.0000	1.0000	1.0000	2694	0.2s	1.0000
CNN	0.9792	0.9791	1.0000	6190	17.2s	0.9375

图 14-1　手写 BPNN 与 CNN 在同一批 6 类合成声纹上的对比：BPNN 参数量 2694、CNN 6190，测试准确率 1.000 与 0.979。参数少反而更准，说明在这个数据规模下模型复杂度不是瓶颈

④ 近年变压器/设备声纹文献的做法

下表只收录本讲义实际查阅过的文献，数字为其原文自报结果：

表 14-2　近年变压器与设备声纹文献的做法与结果

文献	特征	模型	数据/对象	原文自报结果
张重远 2020	Mel 时频谱（降维）	CNN	铁芯模型试验台，3 种工况	识别率 99.71%
刘云鹏 2020	50 Hz 倍频倒谱系数	GRU	变压器直流偏磁	偏磁声纹识别（时序建模）
陆云才 2023	时域+频域多通道混合特征	CNN + 集成学习	自建变压器声纹样本库	单一缺陷 99.2%、混合缺陷 99.7%

文献	特征	模型	数据/对象	原文自报结果
李楠 2023	小波声纹特征图谱（4 传感器融合，熵权法加权）	ShuffleNetV2	铁芯松动 25/50/75/100%	平均准确率 99.6%；比 FFT/GAF/MTF/RP 高 12.2%，比单传感器高 5.8%
李嘉宁 2025	可学习 Gabor 卷积替代 MFCC 前端	改进 Transformer（多头+时频注意力）	GIS 操作机构，含噪	高信噪比 96%；10 dB 时提升 >9.4%
齐子豪 2025	多粒度知识特征	Transformer	变压器故障声纹	故障声纹辨识

⑤ 从这些工作里能提炼的五条经验

经验 **1**：特征侧往往比模型侧更有效。张重远用 Mel 谱+CNN 就到 99.71%，刘云鹏用"50 Hz 倍频"这个物理先验特征也能解决问题——都不是靠更深的网络。设备声纹的样本量通常只有几百到几千条，堆模型收益有限。

经验 **2**：多传感器融合是稳定增益。李楠 2023 用 4 个传感器、熵权法加权融合图谱，比单传感器高 5.8%。原因很直白：单点测声受驻波和方向性影响，多点能平均掉这些偶然因素。

经验 **3**：噪声是主要敌人，且必须单独报告。李嘉宁 2025 特意报告"10 dB 时提升 9.4%"，而不是只报高信噪比的 96%。这正是第 10 讲强调的"鲁棒性曲线"思路。

经验 **4**：前端可以学。李嘉宁用"可学习参数的 Gabor 卷积"替代固定的 MFCC 前端，本质是让网络自己决定滤波器形状——这是 MFCC 之后的一个重要演进方向，也解释了为什么"MFCC 不一定最优"。

经验 **5**：所有 **99%** 都不能横向比较。张重远的 99.71%（3 类、试验台）、陆云才的 99.2%/99.7%（自建库）、李楠的 99.6%（4 级松动程度）——类别数、数据来源、划分方式都不同。把它们排成排行榜是错的，只能看"各自解决了什么问题"。

误区与注意事项

> **误区** ① 以为统计池化只是"求平均"。标准差那一半同样重要，它携带"波动程度"信息。② 在闭集任务上用 **triplet** 损失。没必要，softmax 更快更好。③ 照抄论文的超参数。数据规模差一个数量级，学习率、层数都要重调。④ 把不同论文的准确率排榜。数据集条件不同，比较无意义。

自检

- [ ] 我能写出统计池化的均值+标准差公式，并解释它为什么能处理变长输入。
- [ ] 我能说清 AAM-softmax 相比 softmax 多了什么约束、为什么开集场景需要它。
- [ ] 我能说出至少三条从文献里提炼的工程经验。
- [ ] 我知道为什么这些论文的 99% 不能横向比较。

第 15 讲

收尾：学习路线回顾与进阶方向

完整依赖链回顾

表 15-1 完整依赖链回顾

环节	核心问题	关键公式/数字	对应讲次
物理	声音是什么	$x(t)=A\cdot\sin(2\pi ft+\varphi)$；变压器基频 100 Hz	1
数字化	怎么变成数	$F_s>2f_{max}$；SNR≈6.02B+1.76；25 ms/10 ms	2
时域	响度/快慢/周期	$E=\Sigma x^2$；$Z=2fN/F_s$；$f_0=F_s/k_0$	3
频域	能量在哪些频率	$X[k]=\Sigma x\cdot e^{-j2\pi kn/N}$；$\Delta f=F_s/N$	4
时频	什么时候出现什么频率	$\Delta t\cdot\Delta f\approx 1$；25 ms→40 Hz	5
特征	怎么压成低维向量	Mel 26 通道；13+13+13=39 维	6
多分辨	瞬态怎么抓	$f\approx F_s/a$；3 层 DWT；IMF	7
模型	怎么从数据学规律	$\delta_2=p-y$；$\partial L/\partial W=a^T\delta$	8–9
评估	怎么知道模型好不好	P/R/F1；AUC；噪声衰减	10
判决	阈值定在哪	EER；2×EER；C_{det}	11–12
经典/深度	用什么模型	GMM-UBM；i-vector；x-vector	13–14

怎么验证自己"真的会了"

三关，从易到难，缺一不可：

> 第一关（能讲）：合上讲义，向别人解释"为什么 MFCC 要取 log"。
>
> 讲不清 = 只是记住了流程，没理解原因。
>
> 第二关（能写）：不看现成代码，写出能通过 1e-10 对拍的 DFT。
>
> 写不出 = 公式还停在纸面上。
>
> 第三关（能改）：给定一个新需求（比如"要在 10 dB 噪声下识别局放"），
>
> 能说出该改哪个环节、预期会怎样、怎么验证。
>
> 说不出 = 还不会用，只会复现。

第三关才是真正的分水岭。举个具体例子：要在强噪声下识别局放，你会怎么改？可以从四个层面答——数据层（加噪训练、谱减降噪）；特征层（换用对噪声更稳的特征，如谱平坦度、过零率，或学时频掩蔽）；模型层（多传感器融合、注意力机制）；评估层（报告噪声衰减曲线而不是单一准确率）。能把这四层串起来讲，就说明整条链路通了。

进阶方向（结合综述列出的挑战）

龚文洁等 2026 的综述把领域瓶颈列为五条：噪声鲁棒性、样本失衡、可解释性、标准化、跨模态融合。对应到可以动手的题目：

表 15-2　五个挑战与可做的题目

挑战	可做的题目	为什么有价值
噪声鲁棒性	在 0–20 dB 信噪比下系统评测 MFCC / 谱图 / 可学习前端三种方案的衰减曲线	直接对应现场最常见的失败模式
样本失衡	故障样本只有几十条时，比较重采样、代价敏感、少样本学习	现场故障本来就稀有
可解释性	把 CNN 的注意力热图叠回语谱图，看它是否真的在看谐波/脉冲	运维人员不信黑箱
标准化	统一采集协议 + 公开基准数据集	目前各研究各用私有数据，结果无法横比
跨模态	声纹 + 振动 + 温度的多源融合	单一声学通道信息量有限

如果只选一个，我建议做"统一评测"类的工作：把现有几种方法放在同一套数据、同一套指标下比较，并公开代码与数据划分。这类工作技术上不难，但对领域的价值比"再刷一个 99.x%"高得多——因为现在最缺的正是可比性。

最终自检（12 项总验收）

- [ ] 我能不看笔记写出正弦表达式，并说出三个物理量对应什么听感。
- [ ] 我能算出任意采样率、任意帧长下的点数与帧数。
- [ ] 我能解释混叠，并算出一个具体例子。
- [ ] 我能手算 4 点 DFT，并解释"投影"的含义。
- [ ] 我能算出给定窗长的 Δf，并说出它对分辨能力的影响。
- [ ] 我能默写 MFCC 七步，并说出每步的理由。
- [ ] 我能说出小波、EMD 各自解决什么问题、代价是什么。
- [ ] 我能推出 BP 的两条核心公式，并做一次梯度检查。
- [ ] 我能算出卷积的输出尺寸与参数量。
- [ ] 我能识别数据泄漏，并说出三种以上的过拟合对策。
- [ ] 我能解释 EER 与 2×EER，并用 DCF 比较两个阈值。
- [ ] 我能对一篇声纹论文提出至少两个方法学质疑。

附 录

中英术语对照、章节映射与参考文献

常用术语对照

表 A-1　常用术语中英对照

中文	英文	缩写	一句话解释
声纹识别 / 说话人识别	speaker recognition / voiceprint recognition	—	用声音判断"是谁"
声纹验证	speaker verification	—	1:1 判断"是不是这个人"
声纹辨认	speaker identification	—	1:N 判断"是库里哪个人"
短时傅里叶变换	short-time Fourier transform	STFT	分帧做 FFT，得到时频图
梅尔倒谱系数	mel-frequency cepstral coefficients	MFCC	按人耳刻度压成的低维特征
感知线性预测	perceptual linear prediction	PLP	另一套听觉模型特征
等错误率	equal error rate	EER	FA=FR 时的错误率
错误接受率 / 错误拒绝率	false accept rate / false reject rate	FAR / FRR	误受 / 误拒
检测代价函数	detection cost function	DCF	把两类错误代价写进指标
高斯混合模型	Gaussian mixture model	GMM	多个高斯加权描述分布
通用背景模型	universal background model	UBM	描述"所有人"的通用分布
最大后验自适应	maximum a posteriori	MAP	从 UBM 微调到个人模型
身份向量	identity vector	i-vector	低秩总变化空间的坐标
嵌入码	embedding	x-vector / d-vector	神经网络输出的定长身份向量
概率线性判别分析	probabilistic linear discriminant analysis	PLDA	把身份与信道因子分开
支持向量机	support vector machine	SVM	最大间隔线性分类器
联合因子分析	joint factor analysis	JFA	i-vector 的前身
经验模态分解	empirical mode decomposition	EMD	按时间尺度自适应分解
本征模态函数	intrinsic mode function	IMF	EMD 分出的单尺度分量
离散小波变换	discrete wavelet transform	DWT	二进尺度的多分辨分解
倒谱均值归一化	cepstral mean normalization	CMN	减掉信道造成的加性偏移
加性角度间隔	additive angular margin	AAM-softmax	加大类间角度间隔的损失

中文	英文	缩写	一句话解释
统计池化	statistical pooling	—	用均值+标准差把变长压成定长
信噪比	signal-to-noise ratio	SNR	信号功率与噪声功率之比（dB）

与两本教材的章节映射

表 A-2　与两本教材的章节映射

本讲义	韩纪庆《语音信号处理》	王泉《声纹技术》
第 1 讲	2.1 语音产生、2.2 感知	1 声纹前世今生、2.2 声学基础
第 2 讲	3.1 数字化、3.1.2 加窗	2.3 采样量化、2.4 短时分析
第 3 讲	3.2 时域分析	2.5.1 时域特征
第 4 讲	3.3.2 傅里叶频谱分析	2.5.2 傅里叶变换
第 5 讲	3.4 时频分析、3.5 Gabor	2.5.2 STFT
第 6 讲	3.7.4 Mel 倒谱系数	2.5.5 MFCC
第 7 讲	3.6 小波变换	（EMD 为项目补充）
第 8–9 讲	6.10 深度学习语音识别	3.4 深度学习声纹
第 10 讲	（通用机器学习）	3.5 数据处理
第 11 讲	7.1 说话人识别概述	3.1 声纹识别核心
第 12 讲	7.2 阈值选取	3.2 评价指标
第 13 讲	7.2 GMM/UBM、7.5 i-vector	3.3 深度学习前方法
第 14 讲	7.5.2 基于 DNN 的 i-vector	3.4 深度学习、3.4.4 损失函数

说明：本讲义由两书核心知识重新组织编写，配合 `acoustic-lab` 复现项目使用。原书公式多为图片，本讲义公式按标准形式重写并注明出处；"伽马变换/Gammatone"与"EMD/经验模态分解"经全书检索确认两书均无，已明确标注为项目补充。

参考文献

表 A-3　参考文献

编号	文献
[1]	韩纪庆, 张磊, 郑铁然. 语音信号处理（第 3 版）. 北京: 清华大学出版社.
[2]	王泉. 声纹技术：从核心算法到工程实践.
[3]	龚文洁, 林国松, 韦晓广. 变压器声纹故障诊断研究综述. 电子与信息学报, 2026, 48(4): 1591–1607.

编号	文献
[4]	张重远, 罗世豪, 岳浩天, 王博闻, 刘云鹏. 基于 Mel 时频谱-卷积神经网络的变压器铁芯声纹模式识别方法. 高电压技术, 2020, 46(2): 413–423.
[5]	刘云鹏, 王博闻, 岳浩天, 等. 基于 50 Hz 倍频倒谱系数与门控循环单元的变压器偏磁声纹识别. 中国电机工程学报, 2020, 40(14): 4681–4694.
[6]	齐子豪, 仝杰, 张中浩, 等. 基于多粒度知识特征和 Transformer 网络的电力变压器故障声纹辨识方法. 中国电机工程学报, 2025, 45(4): 1311–1322.
[7]	李嘉宁, 李喆, 盛戈皞. 含噪背景下 GIS 操作机构声纹信号可学习特征提取及辨识方法. 高电压技术, 2025, 51(11): 5516–5525.
[8]	李楠, 马宏忠, 段大卫, 等. 基于多传感器融合声纹特征图谱的变压器铁芯松动故障诊断方法. 振动与冲击, 2023, 42(15).
[9]	陆云才, 廖才波, 李群, 等. 基于声纹特征和集成学习的变压器缺陷诊断方法. 电力工程技术, 2023, 42(5): 46–55.
[10]	张瀝. 电力设备故障声学检测技术综述及展望. 2022.

说明：上表只列出本讲义正文实际引用、且已核对过原文的文献。[10] 为综述性文献，用于了解领域全貌；其余为具体方法文献，正文引用处均标注了其原文自报指标。所有自报指标都来自各自的数据集与实验条件，不可横向比较。

常用常数与换算速查

表 A-4　常用常数与换算速查

量	公式 / 数值	示例
频率分辨率	$\Delta f = Fs / N$	8 kHz、256 点 → 31.25 Hz
帧内点数	点数 = 时长(ms)·Fs/1000	16 kHz、25 ms → 400 点
帧数	floor((N−L)/S)+1	16000、L=400、S=160 → 98
量化信噪比	SNR ≈ 6.02B + 1.76 dB	16 bit → 98 dB
过零率与频率	$f = Z \cdot Fs/(2N)$	Z=25、N=200、8 kHz → 500 Hz
自相关求基频	$f_0 = Fs / k_0$	k_0=80、8 kHz → 100 Hz
Mel 刻度	$mel = 2595 \cdot \log_{10}(1+f/700)$	1000 Hz → 1000 mel
小波尺度换算	$f \approx Fs / a$	8 kHz、a=80 → 100 Hz
卷积输出	(H−F+2P)/S+1	32、F=3、P=1、S=1 → 32